Neural Networks 200 (2026) 108860

Contents lists available at ScienceDirect

Neural Networks

journal homepage: www.elsevier.com/locate/neunet

Full Length Article

An efficient wavelet-based physics-informed neural network for multiscale problems

Himanshu Pandey [a], Anshima Singh [a,b], Ratikanta Behera [a,*]

[a] Department of Computational and Data Sciences, Indian Institute of Science, Bangalore, 560012, India
[b] Department of Mathematics, University of Manchester, Oxford Road, Manchester, M13 9PL, United Kingdom

ARTICLE INFO

AMS subject classifications:
65L11
68T07
65T60

Keywords:
Physics-informed neural networks
Wavelets
Singularly perturbed problems
Multiscale problems

ABSTRACT

Physics-informed neural networks (PINNs) are a class of deep learning models that utilize physics in the form of differential equations to address complex problems, including those that may involve limited data availability. However, solving differential equations with rapid oscillations, steep gradients, or singular behavior becomes a challenge for PINNs. To address this, we propose an efficient wavelet-based physics-informed neural network (W-PINN) that learns solutions in wavelet space. Here, we represent the solution in wavelet space using a family of localized wavelets. This framework represents the solution of a differential equation with significantly fewer degrees of freedom while retaining the dynamics of complex physical phenomena. The proposed architecture enables the training process to search for solutions within the wavelet domain, where the multiscale characteristics are less pronounced compared to the physical domain. This facilitates more efficient training for this class of problems. Furthermore, the proposed model does not rely on automatic differentiation (AD) for derivatives involved in the loss function and does not require any prior information regarding the behavior of the solution, such as the location of abrupt features. The removal of the AD requirement significantly reduces training time while maintaining accuracy. Thus, through a strategic fusion of wavelets with PINNs, W-PINNs excel at capturing localized non-linear information, making them well-suited for problems showing abrupt behavior in certain regions, such as singularly perturbed and other multiscale problems. We further analyze the convergence behavior of W-PINN through a comparative study using the Neural Tangent Kernel (NTK) theory. The efficiency and accuracy of the proposed neural network model are demonstrated in various problems, i.e., the FitzHugh-Nagumo (FHN) model, the Helmholtz equation, the Maxwell equation, the Allen-Cahn equation, and lid-driven cavity flow, along with other highly singularly perturbed non-linear differential equations.

1. Introduction

The domain of computational science has been profoundly reshaped in recent years by the emergence of artificial intelligence. Among these developments, physics-informed neural networks (PINNs) (Lagaris et al., 1998; Raissi et al., 2019) have gained substantial recognition as a powerful alternative to traditional numerical methods for solving partial differential equations (PDEs). Traditional numerical methods such as finite difference, finite element, and finite volume approaches have long served as the workhorses of computational science. However, these techniques often struggle with complex geometries that require intricate mesh generation and face significant challenges when scaling to higher-dimensional problems. PINNs address these limitations by offering a mesh-free approach that eliminates the tedious process of mesh construction and the associated numerical artifacts that arise from poor mesh quality (Hu et al., 2024). What makes PINNs particularly compelling is their ability to generalize continuous functional representations across the entire computational domain rather than discrete solution points.

Over the past few years, several variants of PINNs have been proposed to enhance robustness and convergence, including variational physics-informed neural networks (hp-VPINN) (Kharazmi et al., 2021), gradient-enhanced physics-informed neural networks (GPINN) (Yu et al., 2022), and extended physics-informed neural networks (XPINN) (Jagtap & Karniadakis, 2020), and many others to enhance the performance of conventional PINN. For more details, see a comprehensive review of PINNs (Cuomo et al., 2022; Karniadakis et al., 2021) and references therein.

Despite their many significant advantages, the performance of PINNs significantly deteriorates when dealing with problems that exhibit high

* Corresponding author.
E-mail addresses: phimanshu@iisc.ac.in (H. Pandey), anshimasingh@iisc.ac.in, anshima.singh@manchester.ac.uk (A. Singh), ratikanta@iisc.ac.in (R. Behera).

https://doi.org/10.1016/j.neunet.2026.108860
Received 18 September 2025; Received in revised form 20 January 2026; Accepted 13 March 2026
Available online 17 March 2026

gradients, rapid oscillations, or singular behavior (Karniadakis et al., 2021). Wang et al. demonstrate through extensive numerical experiments that conventional PINN methods exhibit significant limitations when applied to multiscale problems (Wang et al., 2021a,b). Singularly perturbed differential equations represent another challenging class, as their solutions exhibit thin transition layers, often adjacent to the domain boundaries. These solutions or their derivatives undergo rapid fluctuations within specific areas while maintaining smooth behavior elsewhere (Roos et al., 2008), which presents particular difficulties for standard PINN approaches. A key reason for these challenges is that various components (residual loss, initial condition and boundary loss) in the loss function have significantly different magnitudes and converge at different rates during training, complicating the optimization process (Farhani et al., 2025; Huang & Alkhalifah, 2022). Furthermore, when tackling multiscale problems characterized by multi-frequency or high-frequency features, the conventional PINN method faces dual challenges: not only does it encounter these loss-term magnitude imbalances, but it also struggles to accurately capture high-frequency functions due to spectral bias (Rahaman et al., 2019). These combined limitations ultimately result in compromised prediction accuracy. These limitations are also theoretically verified by Neural Tangent Kernel (NTK) theory for neural network learning, which was originally formulated by Jacot et al. (2018). Subsequently, Cao et al. (2021) provided a detailed spectral analysis of NTK, and later Wang et al. (2022) provided a similar analysis for PINNs.

The recent literature has proposed several approaches to address these limitations. McClenny et. al. introduced Self-adaptive PINNs (SA-PINNs) (McClenny & Braga-Neto, 2023) that employ trainable weights to dynamically balance loss terms, while Arzani et al. (2023) proposed a BL-PINN approach that splits the domain into inner and outer regions to effectively handle boundary layer problems with singular perturbation. Moreover, Wang et al. developed a practical PINN framework for multiscale problems with multi-magnitude loss terms (MMPINN) (Wang et al., 2024), incorporating specialized architectures and regularization strategies. Complementary approaches include spectral PINNs that incorporate specialized basis functions. Notable examples are Gabor PINN (Abedi et al., 2026; Alkhalifah & Huang, 2024) and physics-informed radial basis networks (PIRBNs) (Bai et al., 2023). These methods integrate oscillatory or localized basis functions directly into the network architecture. This integration enhances the network's ability to approximate high-frequency components and localized features in the solution. Although these methods demonstrate improved performance, they still face computational overhead from automatic differentiation (AD) and require careful tuning of multiple hyperparameters. However, AD provides machine-precision derivatives, it comes at the cost of maintaining computation-intensive computational graphs that grow increasingly complex with network depth and order of derivatives involved.

To reduce the overhead associated with AD, several alternative non-AD approaches have been proposed. Monte Carlo approximation techniques for second-order PDE derivatives were explored by Sgouralis and Spiliopoulos (Sirignano & Spiliopoulos, 2018). Navaneeth et al. developed a gradient-free learning methodology utilizing random projections, successfully modeling complex fourth-order phase field fracture scenarios (Navaneeth & Chakraborty, 2023). Other developments include using meshless radial basis functions by Xiao et al. (2023) to compute spatial derivatives, which theoretically accommodates random collocation point arrangements. One more recent development in this direction is DT-PINN (Sharma & Shankar, 2022), which achieved two to four times speedups over AD-based PINNs by eliminating AD through meshless radial basis function-finite differences (RBF-FD). These methods help mitigate computational inefficiencies and improve accuracy. However, derivative-based schemes can encounter difficulties when applied to irregular geometries, higher-order PDEs, or strongly localized features, and often require auxiliary collocation points outside the physical domain. Consequently, designing a framework that is simultaneously efficient, robust, and multiscale-aware remains an open challenge.

Building upon these foundations, the motivation for our present work is two-fold. First, we seek to overcome the challenge of multi-magnitude residuals, which limit the effectiveness of conventional PINNs in capturing sharp transitions and high-frequency features in multiscale problems. Second, we aim to reduce the significant computational overhead associated with AD techniques that dominate existing implementations. To address these challenges simultaneously, we propose a wavelet-based physics-informed neural network (W-PINN) that reformulates physics-informed learning in a multiresolution function space, rather than modifying network architectures or loss-balancing strategies, or heuristic regularization.

The foundational works of Mallat (Mallat, 1989) and Daubechies (Daubechies, 1992) on wavelet analysis, demonstrate that functions exhibiting sharp gradients, oscillations, or localized singularities, often admit compact and well-conditioned representations in wavelet space. In the context of physics-informed learning, this property is particularly advantageous, as conventional PINNs approximate solutions pointwise in the physical domain, where multiscale features lead to severe loss imbalance and slow convergence of high-frequency modes, a wavelet representation decomposes the solution into scale-separated components. As a result, localized or high-frequency structures that are difficult for neural networks to learn directly become isolated within a small subset of wavelet coefficients, enabling W-PINNs to resolve boundary layers, sharp transitions, and oscillatory features more efficiently. Incorporation of wavelets into PINNs has been attempted in Uddin et al. (2023). However, the authors of Uddin et al. (2023) focused on wavelets solely as activation functions, with the overall structure of PINNs remaining unchanged from its original introduction. Our findings show that this method also fails to satisfactorily approximate the problems under consideration.

In this study, the proposed W-PINN employs wavelets as a solution basis, representing the unknown solution as a linear combination of multi-resolution wavelet functions whose coefficients are learned by the neural network. Unlike Gabor PINN and PIRBN, which enhance approximation while remaining physical-space and AD-dependent, this formulation shifts the learning task from pointwise solution approximation to coefficient learning in wavelet space, where multiscale features are naturally separated across resolutions. The proposed neural architecture does not rely on the automatic differentiation to compute the derivatives involved in the loss function, thereby significantly reducing training time. In addition, this method does not require prior information about the nature of the solution, making it practical and easy to implement. We used Gaussian and Mexican hat wavelets for implementation, and their performance was compared with conventional PINN and state-of-the-art methods to validate the efficacy. Our approach demonstrates high accuracy across a range of differential equations that exhibit steep gradients, rapid oscillations, singularities, and multiscale behavior, establishing it as a robust method for these classes of problems. The key contributions of this work can be summarized as follows.

- We introduce a W-PINN framework that eliminates automatic differentiation in loss function derivative computation, significantly accelerating training while maintaining or improving solution accuracy compared to recent methods in the literature.
- W-PINN effectively addresses the loss balancing challenges inherent in conventional PINNs, particularly for problems that exhibit multiscale phenomena, singular behavior, or rapid oscillations in their solutions.
- The proposed method is validated through NTK theory and various examples, including the FHN model, Helmholtz equation, Allen-Cahn equation, Maxwell equation, Lid-driven cavity flow, and various other singularly perturbed problems.

The organization of the remaining paper is summarized as follows: Section 2 initiates with the introduction of standard PINNs, a summary of NTK theory for PINNs, followed by a thorough discussion on the design and operational mechanism of W-PINNs. Section 3 presents

numerical results along with a comparison with other well-known methods in the literature for various differential equations. In this section, we also justify a much faster convergence of W-PINN using NTK theory. Finally, Section 4 serves as the concluding segment, offering a concise summary of the key findings, contributions, and future work along with limitations.

2. Methodology

2.1. Physics-informed neural networks (PINNs)

A neural network is a mathematical model consisting of layers of neurons interconnected via non-linear operations. These layers include an input layer for initial data, multiple hidden layers for complex computations, and an output layer for predictions. It is widely acknowledged that with enough hidden layers and neurons per layer, a neural network can approximate any function (Cybenko, 1989). The values of neurons in each layer depend on the previous layer in the following manner:

$$\begin{aligned} &\text{Input layer:} && \boldsymbol{z}^0 = \mathbf{x} \in \mathbb{R}^{n_0}, \\ &\text{Hidden layers:} && \boldsymbol{z}^k = \sigma(\boldsymbol{\omega}^k \boldsymbol{z}^{k-1} + \boldsymbol{b}^k), 1 \le k \le L-1, \\ &\text{Output layer:} && \boldsymbol{z}^L = \boldsymbol{\omega}^L \boldsymbol{z}^{L-1} + \boldsymbol{b}^L \in \mathbb{R}^{n_L}, \end{aligned} \tag{1}$$

where $\boldsymbol{z}^k \in \mathbb{R}^{n_k}$ is the outcome of k-th layer, $\boldsymbol{\omega}^k \in \mathbb{R}^{n_k \times n_{k-1}}$, $\boldsymbol{b}^k \in \mathbb{R}^{n_k}$, and n_k is the number of neurons in k^{th} layer. Here, $\boldsymbol{\omega}$ and $\boldsymbol{b}$ denote the weight and bias, the model parameters to be optimized. Moreover, σ is a non-linear mapping known as an activation function. This process is widely known as feed-forward propagation. We define a loss function ($\mathcal{L}$) for network output, which quantifies the disparity between the network's predictions and the desired outcomes. The neural network is trained using backpropagation (Rumelhart et al., 1986), a technique wherein optimization algorithms, such as gradient descent, are used to minimize the loss function and fine-tune the model parameters $(\boldsymbol{\omega}, \boldsymbol{b})$. Trained parameters capture the underlying structure of the problem under consideration. A well-known class of neural networks is physics-informed neural networks (PINNs), which incorporate the underlying physics of the system by using a governing set of equations along with initial and boundary conditions into the loss function. It measures how much the network's predicted solution violates the differential equation at several collocation points.

Consider a PINN model, $\hat{u}(\boldsymbol{x}, t; \boldsymbol{\theta})$ that predicts the solution of a differential equation in the spatiotemporal domain, $\boldsymbol{x} \in \Omega$ and $t \in (0, T]$. Here $\boldsymbol{\theta}$ are trainable parameters of the network, which describe the non-linear relationship $(\boldsymbol{x}, t) \to \hat{u}$ according to (1). A general form of a partial differential equation (PDE) can be given by

$$\begin{cases} \mathscr{L}_{\boldsymbol{x},t}[u(\boldsymbol{x},t)] = f(\boldsymbol{x},t), & \boldsymbol{x} \in \Omega, t \in (0,T], \\ \mathscr{B}[u(\boldsymbol{x},t)] = g(\boldsymbol{x},t), & \boldsymbol{x} \in \partial\Omega, t \in (0,T], \\ \mathscr{I}[u(\boldsymbol{x},0)] = h(\boldsymbol{x}), & \boldsymbol{x} \in \Omega, \end{cases} \tag{2}$$

where $\mathscr{L}_{\boldsymbol{x},t}[\cdot]$ represents differential operator, $\mathscr{B}[\cdot]$ and $\mathscr{I}[\cdot]$ correspond to boundary and initial conditions respectively. Let $\{(\boldsymbol{x}_i^r, t_i^r)\}_{i=1}^{N_r} \subset \Omega \times (0,T]$ denote a set of collocation points used to evaluate the governing differential operator. Similarly, let $\{(\boldsymbol{x}_i^{bc}, t_i^{bc})\}_{i=1}^{N_{bc}} \subset \partial\Omega \times (0,T]$ and $\{\boldsymbol{x}_i^{ic}\}_{i=1}^{N_{ic}} \subset \Omega$ denote the spatial collocation points corresponding to the boundary and initial conditions. The loss function for PINN can be defined as $\mathcal{L} = \mathcal{L}_{\text{res}} + \mathcal{L}_{\text{ic}} + \mathcal{L}_{\text{bc}}$. Here,

$$\begin{aligned} \mathcal{L}_{\text{res}} &= \frac{1}{N_r} \sum_{i=1}^{N_r} \left(\mathscr{L}\left[\hat{u}(\boldsymbol{x}_i^r, t_i^r; \boldsymbol{\theta})\right] - f(\boldsymbol{x}_i^r, t_i^r) \right)^2, \\ \mathcal{L}_{\text{bc}} &= \frac{1}{N_{bc}} \sum_{i=1}^{N_{bc}} \left(\mathscr{B}\left[\hat{u}(\boldsymbol{x}_i^{bc}, t_i^{bc}; \boldsymbol{\theta})\right] - g(\boldsymbol{x}_i^{bc}, t_i^{bc}) \right)^2, \\ \mathcal{L}_{\text{ic}} &= \frac{1}{N_{ic}} \sum_{i=1}^{N_{ic}} \left(\hat{u}(\boldsymbol{x}_i^{ic}, 0; \boldsymbol{\theta}) - h(\boldsymbol{x}_i^{ic}) \right)^2, \end{aligned} \tag{3}$$

where $\mathcal{L}_{\text{res}}$ denotes the residual mean squared error, $\mathcal{L}_{\text{bc}}$ and $\mathcal{L}_{\text{ic}}$ denote mean squared errors for boundary and initial conditions, respectively. During optimization, the derivative of the loss function with respect to the model parameters and derivatives of the network's output with respect to input parameters, exhibited in the loss function, are required. These derivatives are effectively computed using "automatic differentiation" (Baydin et al., 2017). Thus, the objective of PINNs is to identify the optimal parameters that minimize the specified loss function, which encapsulates all physics-related information.

2.2. Neural tangent kernel for PINNs

The Neural Tangent Kernel (NTK) theory offers an effective mathematical formulation for analyzing the dynamics of neural network functions, $f(\boldsymbol{x}; \boldsymbol{\theta})$, where $\boldsymbol{x}$ is input and $\boldsymbol{\theta}$ are the network's parameters. NTK was first proposed by Jacot et al. (2018). They introduced a kernel regression framework that allows the study of neural network training in the function space rather than in the parameter space. Since the loss function is defined as a squared L_2-error between the network output and a target function, the resulting optimization problem is convex with respect to the network output $f(\boldsymbol{x})$, even though it remains highly non-convex with respect to the parameters $\boldsymbol{\theta}$, this makes analysis more tractable in function space. Jacot et al. demonstrated that, under training with an infinitesimally small learning rate, the network function $f(\boldsymbol{x}; \boldsymbol{\theta})$ evolves according to the kernel gradient descent with respect to the NTK. Moreover, in the infinite-width limit, the NTK converges to a deterministic limiting kernel that remains constant throughout training.

Subsequent work by Cao et al. (2021) provided a spectral analysis of the NTK, showing that smaller NTK eigenvalues lead to slower convergence of high-frequency components in the target function. This phenomenon, known as the "spectral bias" of neural networks, states that neural networks tend to learn low-frequency features more efficiently than high-frequency ones. Building on this foundation, Wang et al. (2022) applied NTK theory to PINNs, analyzing how the NTK spectrum governs the convergence rate of the training error. Their analysis can be summarized as follows. Consider a PINN $\hat{u}(\boldsymbol{x}; \boldsymbol{\theta})$ that is an approximate solution of the following PDE.

$$\begin{cases} \mathscr{L}[u(\boldsymbol{x})] = f(\boldsymbol{x}), \boldsymbol{x} \in \Omega, \\ \mathscr{B}[u(\boldsymbol{x})] = g(\boldsymbol{x}), \boldsymbol{x} \in \partial\Omega. \end{cases}$$

Here, $\mathscr{L}$ and $\mathscr{B}$ represent differential operators and boundary conditions, respectively. For given data points $\left\{\boldsymbol{x}_r^i, f(\boldsymbol{x}_r^i)\right\}_{i=1}^{N_r}$ and $\left\{\boldsymbol{x}_b^i, g(\boldsymbol{x}_b^i)\right\}_{i=1}^{N_b}$ the dynamics of $\hat{u}$ and $\mathscr{L}[\hat{u}]$ follows.

$$\begin{aligned} \begin{bmatrix} \frac{d\hat{u}(\boldsymbol{x}_b;\boldsymbol{\theta}(n))}{dn} \\ \frac{d\mathscr{L}[\hat{u}](\boldsymbol{x}_r;\boldsymbol{\theta}(n))}{dn} \end{bmatrix} &= -\begin{bmatrix} \boldsymbol{K}_{uu}(n) & \boldsymbol{K}_{ur}(n) \\ \boldsymbol{K}_{ru}(n) & \boldsymbol{K}_{rr}(n) \end{bmatrix} \begin{bmatrix} \hat{u}(\boldsymbol{x}_b;\boldsymbol{\theta}(n)) - g(\boldsymbol{x}_b) \\ \mathscr{L}[\hat{u}](\boldsymbol{x}_r;\boldsymbol{\theta}(n)) - f(\boldsymbol{x}_r) \end{bmatrix} \\ &= -\boldsymbol{K}(n) \begin{bmatrix} \hat{u}(\boldsymbol{x}_b;\boldsymbol{\theta}(n)) - g(\mathbf{x}_b) \\ \mathscr{L}[\hat{u}](\boldsymbol{x}_r;\boldsymbol{\theta}(n)) - f(\boldsymbol{x}_r) \end{bmatrix}, \end{aligned} \tag{4}$$

where n is training step and $K(n)$ is NTK for PINN, whose $\{i, j\}$-th element is obtained by:

$$\begin{aligned} (\boldsymbol{K}_{uu})_{ij}(n) &= \left\langle \frac{\partial \hat{u}(\boldsymbol{x}_b^i;\boldsymbol{\theta}(n))}{\partial \boldsymbol{\theta}}, \frac{\partial \hat{u}(\boldsymbol{x}_b^j;\boldsymbol{\theta}(n))}{\partial \boldsymbol{\theta}} \right\rangle, \\ (\boldsymbol{K}_{ur})_{ij}(n) &= \left\langle \frac{\partial \hat{u}(\boldsymbol{x}_b^i;\boldsymbol{\theta}(n))}{\partial \boldsymbol{\theta}}, \frac{\partial \mathscr{L}[\hat{u}](\boldsymbol{x}_r^j;\boldsymbol{\theta}(n))}{\partial \boldsymbol{\theta}} \right\rangle, \\ (\boldsymbol{K}_{rr})_{ij}(n) &= \left\langle \frac{\partial \mathscr{L}[\hat{u}](\boldsymbol{x}_r^i;\boldsymbol{\theta}(n))}{\partial \boldsymbol{\theta}}, \frac{\partial \mathscr{L}[\hat{u}](\boldsymbol{x}_r^j;\boldsymbol{\theta}(n))}{\partial \boldsymbol{\theta}} \right\rangle, \end{aligned}$$

where $\langle \cdot, \cdot \rangle$ represents the inner product in all parameters. Similarly to NTK theory for neural networks, Wang et al. (2022) establish that under certain assumptions, NTK for a PINN also remains constant during training; therefore, we approximate $\boldsymbol{K}(n) \approx \boldsymbol{K}(0)$. This assumption leads

to a solution of (4) as

$$\begin{bmatrix} \hat{u}(\boldsymbol{x}_b;\boldsymbol{\theta}(n)) \\ \mathscr{L}[\hat{u}](\boldsymbol{x}_r;\boldsymbol{\theta}(n)) \end{bmatrix} = \left(\boldsymbol{I} - e^{-\boldsymbol{K}(0)n}\right) \begin{bmatrix} g(\boldsymbol{x}_b) \\ \\ f(\boldsymbol{x}_r) \end{bmatrix}, \tag{5}$$

equivalently, $$\begin{bmatrix} \hat{u}(\boldsymbol{x}_b;\boldsymbol{\theta}(n)) \\ \mathscr{L}[\hat{u}](\boldsymbol{x}_r;\boldsymbol{\theta}(n)) \end{bmatrix} - \begin{bmatrix} g(\boldsymbol{x}_b) \\ f(\boldsymbol{x}_r) \end{bmatrix} = -e^{-\boldsymbol{K}(0)n} \begin{bmatrix} g(\boldsymbol{x}_b) \\ f(\boldsymbol{x}_r) \end{bmatrix}.$$

This is how the training error evolves with iterations. It can be further simplified using the semi-positive definiteness of NTK. We decompose $\boldsymbol{K}(0)$ as $\boldsymbol{K}(0) = \boldsymbol{Q}^T \boldsymbol{\Lambda} \boldsymbol{Q}$, where $\boldsymbol{Q}$ is an orthogonal matrix, and $\boldsymbol{\Lambda}$ is a diagonal matrix with eigenvalues $\lambda_i \geq 0$ as diagonal elements. With this, we rewrite (5) as

$$\begin{bmatrix} \hat{u}(\boldsymbol{x}_b;\boldsymbol{\theta}(n)) \\ \mathscr{L}[\hat{u}](\boldsymbol{x}_r;\boldsymbol{\theta}(n)) \end{bmatrix} - \begin{bmatrix} g(\boldsymbol{x}_b) \\ f(\boldsymbol{x}_r) \end{bmatrix} = -\boldsymbol{Q}^T e^{-\boldsymbol{\Lambda} n} \boldsymbol{Q} \begin{bmatrix} g(\boldsymbol{x}_b) \\ f(\boldsymbol{x}_r) \end{bmatrix}.$$

This provides an estimate of the convergence of a PINN, which is governed by the magnitude of the eigenvalues of the NTK. Specifically, the larger the eigenvalue λ_i, the faster the i-th error component decays. Detailed mathematics of the above analysis can be found in Wang et al. (2022). Later in the results section, we use these findings to establish a much faster convergence of WPINN over PINN for stiff problems.

2.3. Wavelet-based physics-informed neural networks (W-PINNs)

The proposed W-PINN is designed as an integrated three-module architecture that decouples local feature extraction, multiscale solution representation, and derivative evaluation. Rather than directly approximating the solution in physical space, W-PINN reformulates physics-informed learning as a coefficient identification problem in a pre-computed wavelet basis. This design enables the neural network to focus on learning scale-aware coefficients, while deterministic wavelet operators handle reconstruction and differentiation.

The W-PINN formulation begins by defining a family of wavelet basis functions, constructed as scaled and translated versions of a mother wavelet, which can be written as

$$\Psi_{j,k}(x) = \sqrt{2^j}\psi(2^j x - k), \quad j,k \in \mathbb{Z}.$$

Here, j denotes the scale (or resolution level) and k is the translation parameter. For a compact domain $[a,b]$, the number of wavelet basis functions is determined by a predefined set of resolution levels $J = \{J_1, J_2, \ldots, J_N\}$. For a fixed $j \in J$, the translation indices k span the interval $\left[\lfloor (a-\gamma)\cdot 2^j \rfloor, \lceil (b+\gamma)\cdot 2^j \rceil\right]$, where γ is a translation hyperparameter (of order of domain length), $\lfloor\cdot\rfloor$ and $\lceil\cdot\rceil$ denote floor and ceiling functions, respectively. This ensures sufficient coverage of the domain at each resolution level.

After constructing a family of wavelets, the solution of the differential equation is represented in wavelet space, such that the learning task is shifted from point-wise solution approximation in the physical domain to coefficient learning across multiple resolutions:

$$\hat{u}(x) = \sum_{j=J_1}^{J_N} \sum_{k=\lfloor (a-\gamma)\cdot 2^j \rfloor}^{\lceil (b+\gamma)\cdot 2^j \rceil} c_{j,k} \Psi_{j,k}(x) + \mathcal{B}, \tag{6}$$

where $c_{j,k}$ are trainable coefficients and $\mathcal{B}$ is a trainable bias to allow the representation of nonzero-mean solutions, given that the chosen wavelets are symmetric and inherently zero-mean. Extension of this formulation for higher-dimensional problems is straightforward, for instance, here is a formulation for a 2D problem:

$$\Psi_{j_1,j_2,k_1,k_2}(x,y) = \sqrt{2^{j_1}\cdot 2^{j_2}}\psi_X(2^{j_1}x - k_1)\psi_Y(2^{j_2}y - k_2),$$
$$\hat{u}(x,y) = \sum_{j_1=J_{11}}^{J_{1N_1}} \sum_{j_2=J_{21}}^{J_{2N_2}} \sum_{k_1=\lfloor (a_1-\gamma)\cdot 2^{j_1} \rfloor}^{\lceil (b_1+\gamma)\cdot 2^{j_1} \rceil} \sum_{k_2=\lfloor (a_2-\gamma)\cdot 2^{j_2} \rfloor}^{\lceil (b_2+\gamma)\cdot 2^{j_2} \rceil} c_{j_1,j_2,k_1,k_2} \Psi_{j_1,j_2,k_1,k_2}(x,y) + \mathcal{B}. \tag{7}$$

Here ψ_X and ψ_Y represent 1D wavelet functions in x and y directions, respectively. This hierarchical construction of wavelet families at different resolutions and representing the solution in wavelet space enables capturing features at multiple scales, from broad, global behavior to fine, local details. In this study, we consider three mother wavelets, namely, the Gaussian, the Mexican Hat and the real Morlet. The mathematical expressions for the Mexican hat ($\psi^M(x)$), the Gaussian ($\psi^G(x)$) and the real Morlet ($\psi^{\mathcal{M}}(x)$) can be written as

$$\psi^M(x) = (1-x^2)e^{-\frac{x^2}{2}}, \psi^G(x) = -xe^{-\frac{x^2}{2}}, \psi^{\mathcal{M}}(x) = \cos(x)e^{-\frac{x^2}{2}}.$$

The W-PINN architecture comprises three components, each serving a distinct and complementary role in addressing multiscale representation, optimization stiffness, and computational efficiency:

1. **Pointwise feature mapping**: A shallow fully-connected network that maps spatial-temporal coordinates to a latent representation, capturing local nonlinear interactions without attempting to directly approximate oscillatory or singular solution components.
2. **Global coefficient predictor**: A deep fully-connected network that predicts wavelet coefficients $c_{j,k}$, effectively learning the solution in wavelet space where multiscale features are naturally separated across resolutions.
3. **Non-trainable inverse mapping**: A fixed layer that reconstructs the solution in physical space using predicted coefficients and pre-computed wavelet basis matrices, enabling exact and efficient derivative evaluation.

The architecture of W-PINN for a two-dimensional problem is illustrated in Fig. 1. In general, for a d-dimensional problem, the network architecture comprises an input layer with d neurons corresponding to the spatial-temporal dimensions. These inputs are processed through a shallow network that maps each collocation point to a corresponding entry in the feature layer. For one-dimensional problems, this mapping can be simplified by directly using the input layer as the feature layer. Now, the feature layer passes through a fully connected network to get coefficients. To further enhance model performance, these coefficients can be separately fine-tuned through hyperparameter optimization. The final solution and its derivatives are then computed by combining these optimized coefficients with the pre-calculated wavelet matrices. A trainable bias is also added to the network to incorporate $\mathcal{B}$.

For a given set of collocation points $\{x_0, x_1, \cdots, x_N\}$ and resolution set $J = [J_1, J_2, \cdots, J_n]$, using the domain $[0,1]$ and $\gamma = 0$, the wavelet matrices are constructed as follows:

$$\mathbf{W} = \begin{pmatrix} \psi_{J_1,0}(x_0) & \cdots & \psi_{J_1,2^{J_1}}(x_0) & \cdots & \psi_{J_n,2^{J_n}}(x_0) \\ \psi_{J_1,0}(x_1) & \cdots & \psi_{J_1,2^{J_1}}(x_1) & \cdots & \psi_{J_n,2^{J_n}}(x_1) \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ \psi_{J_1,0}(x_N) & \cdots & \psi_{J_1,2^{J_1}}(x_N) & \cdots & \psi_{J_n,2^{J_n}}(x_N) \end{pmatrix},$$

$$\mathbf{D_1W} = \begin{pmatrix} \psi'_{J_1,0}(x_0) & \cdots & \psi'_{J_1,2^{J_1}}(x_0) & \cdots & \psi'_{J_n,2^{J_n}}(x_0) \\ \psi'_{J_1,0}(x_1) & \cdots & \psi'_{J_1,2^{J_1}}(x_1) & \cdots & \psi'_{J_n,2^{J_n}}(x_1) \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ \psi'_{J_1,0}(x_N) & \cdots & \psi'_{J_1,2^{J_1}}(x_N) & \cdots & \psi'_{J_n,2^{J_n}}(x_N) \end{pmatrix},$$

where $\psi'(x)$ is single derivate of $\psi(x)$ and similarly $\mathbf{D_2W}$ is constructed by double derivative of $\psi(x)$. For a given resolution set J, the number of wavelet basis functions at the resolution level J_i scales as $O(2^{J_i})$. Consequently, the total number of columns in the wavelet matrix, $\mathbf{W}$, grows as $\sum_{j\in J} O(2^j)$. Once the approximate solution and its derivatives are obtained, the loss function is computed as (3).

This wavelet-based formulation eliminates the need for automatic differentiation (AD) in computing PDE residuals, which not only makes training faster by reducing the complexity of the computational graph but also avoids potential numerical instabilities that commonly arise in AD during training, particularly in problems exhibiting high gradients or singularities. The core principle behind W-PINN's efficacy is that abrupt

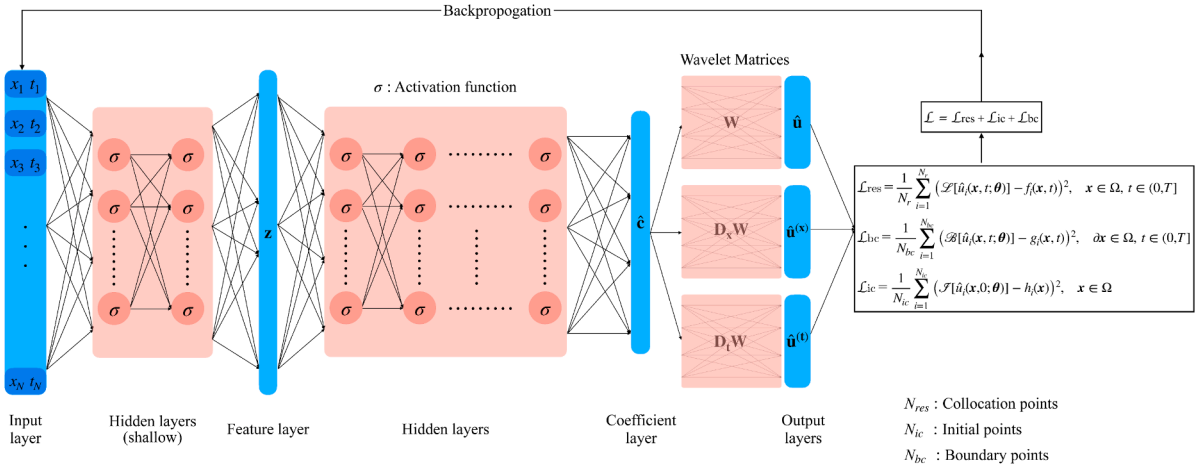

Fig. 1. A schematic W-PINN architecture.

and multiscale features in the physical space become smoother in the wavelet domain, which improves conditioning of the optimization problem, leading to faster convergence and greater robustness in multiscale regimes.

3. Results and discussions

In this section, we present the results obtained using the proposed method over a handful of examples and compare them with the performance of conventional PINNs and state-of-the-art methods. For a fair comparison, all network parameters are kept the same throughout (A.18 and A.19), and are tabulated along with the examples. We utilize the Adam optimizer (Kingma & Ba, 2014), which combines two stochastic gradient descent algorithms: gradient descent with momentum and root mean squared propagation (RMSProp). RMSprop, also known as adaptive gradient descent, adapts the learning rate with iterations. The Adam optimizer is memory efficient and is well-suited for problems with high-dimensional parameter spaces.

The trainable parameters of networks are initialized using Xavier's technique or Glorot initialization (Glorot & Bengio, 2010), which, in contrast to random initialization, initializes parameters in a suitable range to achieve faster convergence during training. During backpropagation, it maintains gradients within a reasonable range (not too low or high), thereby preventing the algorithm from getting stuck in poor local minima. Besides this, it can also reduce the need for extensive hyperparameter tuning, particularly the learning rate. We sampled the training set by Sobol sequencing throughout the domain, as we need efficient exploration of the entire input space for a single training set. For evaluation purposes, we employ the relative L_2 metric over uniform samples across the domain.

$$\text{Relative } L_2 \text{ error} = \sqrt{\frac{\sum_{i=1}^{M}(u_i - \hat{u}_i)^2}{\sum_{i=1}^{M} u_i^2}}, \tag{8}$$

where M represents the total number of testing samples, u_i is the exact solution and $\hat{u}_i$ is the predicted one for the i^{th} sample. In cases where the solution is not available analytically, we consider the exact solution as the numerical solution obtained using an in-house solver.

For implementation purposes, PyTorch 2.4.1 with CUDA 12.1 is used. Training is done on the NVIDIA RTX A6000, which has 48GB of GPU memory. As the performance of a neural network is highly sensitive to the initialization of its parameters, we perform each experiment five times and report the relative L_2-error and average training time based on these five random runs. The source code of the proposed W-PINN method in this work is available on GitHub at https://github.com/himanshup21/W-PINN.git.

Table 1
Parameters used for solving Example 1.

Parameters	Value
Translation hyperparameter γ	1.0
Set of resolutions ($J_M/J_G/J_{\mathcal{M}}$)	[0,8]/[0,10]/[0,9]
Number of hidden layers	6
Neurons per layer	50
Number of collocation points	10^3
Maximum number of iterations	5×10^4

Table 2
Average training time (in seconds) of PINN and W-PINN with network depth for Example 1.

Depth	PINN	W-PINN	Speed-up
2	20.6	7.3	2.8x
4	27.2	6.2	4.4x
8	47.6	8.3	5.7x
16	82.6	12.4	6.6x
32	153.5	22.8	6.7x
64	288.4	38.7	7.4x

Example 1. Consider the following one-dimensional linear advection-diffusion equations (Bender & Orszag, 1999):

$$\begin{cases} \epsilon \dfrac{d^2u}{dx^2} + (1+\epsilon)\dfrac{du}{dx} + u = 0, x \in (0,1), \\ u(0) = 0, u(1) = 1. \end{cases} \tag{9}$$

Here, $0 < \epsilon \ll 1$. The exact solution to the considered problem is

$$u(x) = \frac{\exp(-x) - \exp(-\frac{x}{\epsilon})}{\exp(-1) - \exp(-\frac{1}{\epsilon})}.$$

Such models, called Friedrich's boundary layer models, are often used to show how difficult it is to model viscous flow boundary layers (Arzani et al., 2023). It is impossible for PINNs to capture the singularity that happens when $\epsilon \to 0$.

The parameters of the W-PINN for the Example 1 are listed in Table 1. Table 2 compares the training time of automatic differentiation based PINN and W-PINN. The experiments were conducted with $\epsilon = 2^{-4}$ and each test was executed for 10^4 iterations. The results demonstrate that conventional PINN exhibits a steeper growth in training time with an increase in network depth, requiring up to 7.4 times longer than W-PINN at depth 64. This substantial difference in computational overhead can be attributed to W-PINN's elimination of automatic differentiation calculations, which become increasingly expensive in conventional PINNs

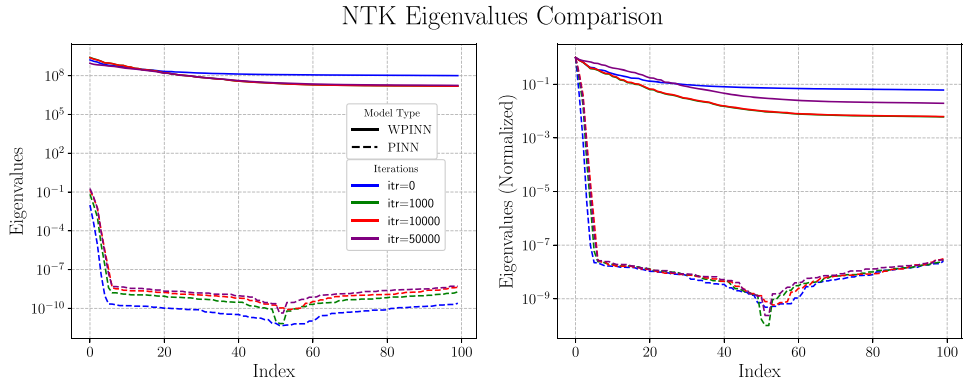

Fig. 2. Comparison of NTK eigenvalues of PINN(dashed line) and W-PINN (solid line) for Example 1 with $\epsilon = 2^{-7}$ at various iterations. Eigenvalues are sorted in descending order and plotted against the respective index.

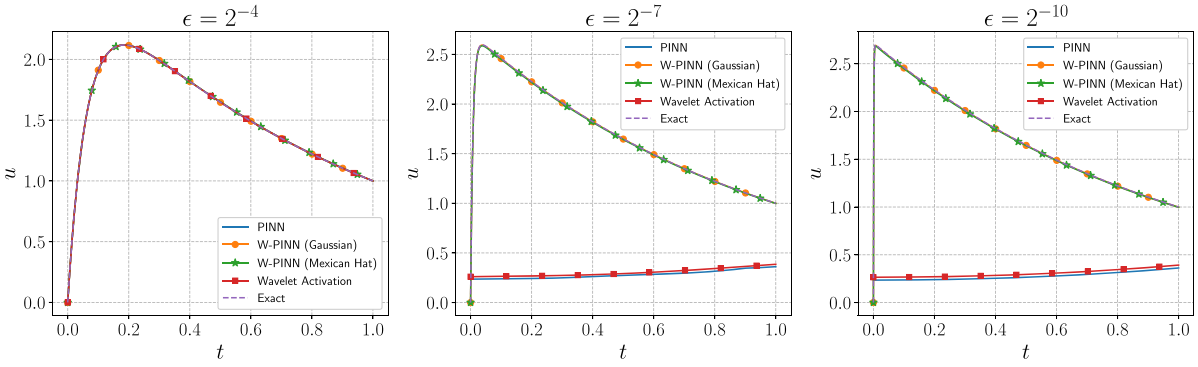

Fig. 3. Comparison of solutions obtained by PINN, with wavelet activation, and W-PINN methods for Example 1. From left to right: with $\epsilon = 2^{-4}$, $\epsilon = 2^{-7}$, and $\epsilon = 2^{-10}$.

Table 3
Performance comparison of various methods for solving Example 1.

ϵ	Method	L_2-error	Average Training Time
$\epsilon = 2^{-4}$	PINN	7.9e-05	1m 58s
	Wavelet Activation (Uddin et al., 2023)	3.9e-05	52s
	PIRBN (Bai et al., 2023)	9.7e-05	2m 21s
	Gabor PINN (Alkhalifah & Huang, 2024)	2.1e-04	3m 15s
	W-PINN (Gaussian)	2.5e-05	23s
	W-PINN (Mexican hat)	8.1e-05	48s
	W-PINN (Morlet)	7.2e-05	1m 33s
$\epsilon = 2^{-7}$	PINN	0.83	-
	Wavelet Activation (Uddin et al., 2023)	0.85	-
	PIRBN (Bai et al., 2023)	0.81	-
	Gabor PINN (Alkhalifah & Huang, 2024)	8.7e-2	4m 17s
	W-PINN (Gaussian)	5.3e-04	18s
	W-PINN (Mexican hat)	9.2e-04	23s
	W-PINN (Morlet)	8.3e-04	1m 41s
$\epsilon = 2^{-10}$	PINN	0.84	-
	Wavelet Activation (Uddin et al., 2023)	0.85	-
	PIRBN (Bai et al., 2023)	0.85	-
	Gabor PINN (Alkhalifah & Huang, 2024)	0.85	-
	W-PINN (Gaussian)	3.1e-03	38s
	W-PINN (Mexican hat)	4.2e-03	42s
	W-PINN (Morlet)	3.1e-03	1m 55s

as network depth increases. Moreover, Fig. 2 compares the Neural Tangent Kernel (NTK) eigenvalue spectra of W-PINN and standard PINN for $\epsilon = 2^{-7}$ across multiple training iterations. The top hundred NTK eigenvalues are sorted in descending order and plotted against their respective indices. The left panel shows the NTK eigenvalues, while the right panel presents the same spectra normalized by their maximum eigenvalue to highlight relative decay behavior. The NTK eigenvalues of standard PINNs exhibit a rapid decay across modes, implying that higher-frequency components are associated with very small eigenvalues and therefore converge extremely slowly. In contrast, the spectrum of W-PINN decays significantly more slowly, indicating improved coupling to higher-frequency modes. This flatter spectral profile suggests that W-PINN can learn multiscale features more efficiently, leading to faster and more stable convergence.

The comparison of various methods has been presented in Table 3, which provides the relative L_2-error and average training time of the W-PINN, conventional PINN, PINN with gaussian-wavelet activation function (Uddin et al., 2023), PIRBN (Bai et al., 2023), and Gabor PINN (Alkhalifah & Huang, 2024). This table shows that the W-PINN approximates the solution with a much smaller L_2-error for lower values of ϵ. Similar observations are drawn from Fig. 3, which demonstrates that when $\epsilon = 2^{-4}$, there is no such sharp singularity near the origin, and all methods approximate the solution well. However, as ϵ decreases, the exact solution of the problem has a strong singularity near the origin. Then, both PINN and PINN with a wavelet activation function are unable to capture the behavior of the solution accurately. In contrast, W-PINN effectively resolves the singularity, with the predicted solution closely following the exact solution.

Example 2. Consider the following highly singularly perturbed non-linear problem (Cen et al., 2014):

$$\begin{cases} \epsilon \dfrac{d^2u}{dt^2} + (3+t)\dfrac{du}{dt} + u^2 - \sin(u) = f, t \in (0,1], \\ u(0) = 1, u'(0) = 1/\epsilon, \end{cases} \tag{10}$$

where f is chosen such that $u(t) = 2 - \exp(-t/\epsilon) + t^2$ is the exact solution.

This example corresponds to a highly singularly perturbed non-linear equation with a known solution. For Example 2, the network parameters are given in Table 4.

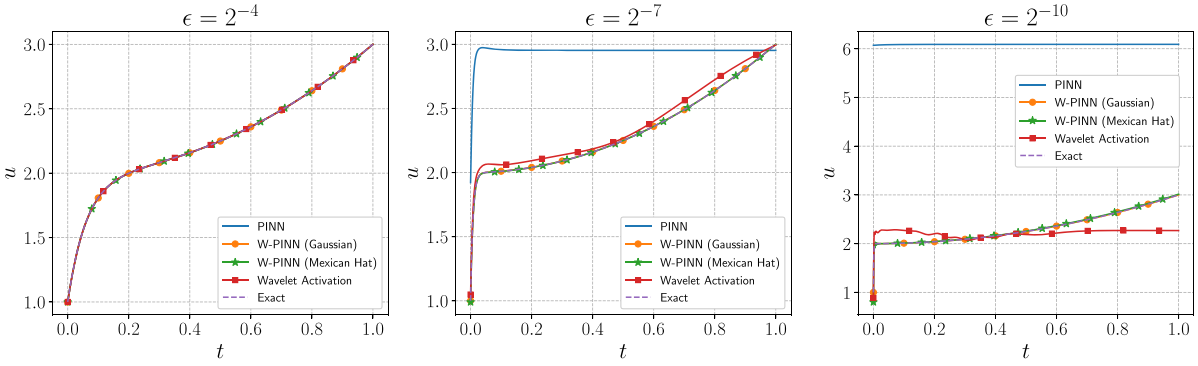

Fig. 4. Comparison of solutions obtained by PINN, PINN with wavelet activation, and W-PINN methods for Example 2. From left to right: with $\epsilon = 2^{-4}$, $\epsilon = 2^{-7}$, and $\epsilon = 2^{-10}$.

Table 4
Parameters used for solving Example 2.

Parameters	Value
Translation hyperparameter γ	1.0
Set of resolutions ($J_M/J_G/J_{\mathcal{M}}$)	[0,9]/[0,10]/[0,10]
Number of hidden layers	6
Neurons per layer	50
Number of collocation points	10^4
Maximum number of iterations	10^5

Table 5
Performance comparison of various methods for solving Example 2.

ϵ	Method	L_2-error	Average Training Time
$\epsilon = 2^{-4}$	PINN	1.2e-04	31s
	Wavelet Activation (Uddin et al., 2023)	2.1e-04	58s
	PIRBN(Bai et al., 2023)	4.7e-05	85s
	Gabor PINN(Alkhalifah & Huang, 2024)	3.3e-04	55s
	W-PINN (Gaussian)	2.3e-05	48s
	W-PINN (Mexican hat)	1.8e-04	51s
	W-PINN (Morlet)	4.1e-04	1m 53s
$\epsilon = 2^{-7}$	PINN	8.5e-02	3m 33s
	Wavelet Activation (Uddin et al., 2023)	2.2e-02	3m 14s
	PIRBN (Bai et al., 2023)	6.6e-04	2m 5s
	Gabor PINN(Alkhalifah & Huang, 2024)	7.8e-03	3m 17s
	W-PINN (Gaussian)	8.3e-05	2m 21s
	W-PINN (Mexican hat)	3.7e-04	2m 24s
	W-PINN (Morlet)	7.4e-05	2m 31s
$\epsilon = 2^{-10}$	PINN	1.4	-
	Wavelet Activation (Uddin et al., 2023)	0.34	-
	PIRBN (Bai et al., 2023)	1.6e-02	3m 13s
	Gabor PINN (Alkhalifah & Huang, 2024)	9.1e-02	3m 42s
	W-PINN (Gaussian)	6.6e-05	2m 27s
	W-PINN (Mexican hat)	4.5e-04	2m 48s
	W-PINN (Morlet)	5.3e-05	2m 13s

As ϵ decreases, the initial layer becomes more pronounced, leading to increased stiffness and multiscale complexity, which becomes challenging to resolve. This behavior is reflected in Table 5, where the proposed W-PINN consistently maintains strong approximation capability across all values of ϵ for different wavelet choices, including Gaussian, Mexican hat, and Morlet wavelets. In contrast, the accuracy of competing approaches such as PINN with a Gaussian wavelet as activation function (Uddin et al., 2023), PIRBN (Bai et al., 2023), and Gabor PINN (Alkhalifah & Huang, 2024) deteriorates as ϵ decreases, while W-PINN remains stable. These results highlight the effectiveness of wavelet-based representations in capturing fine-scale features induced by small ϵ.

Fig. 4 demonstrates similar observations, for $\epsilon = 2^{-10}$, both conventional PINN and PINN with the Gaussian wavelet as activation function (Uddin et al., 2023) are unable to detect the singularity satisfactorily, whereas W-PINN effectively handles the singularity.

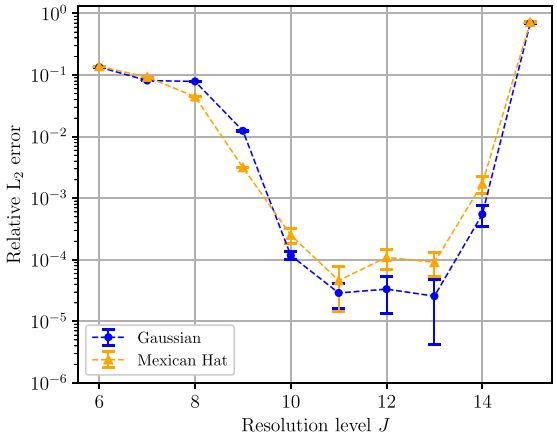

Fig. 5. A plot for relative L_2-error with different wavelet resolution levels ($[0, J]$) for Gaussian (blue) and Mexican-Hat (orange) wavelets.

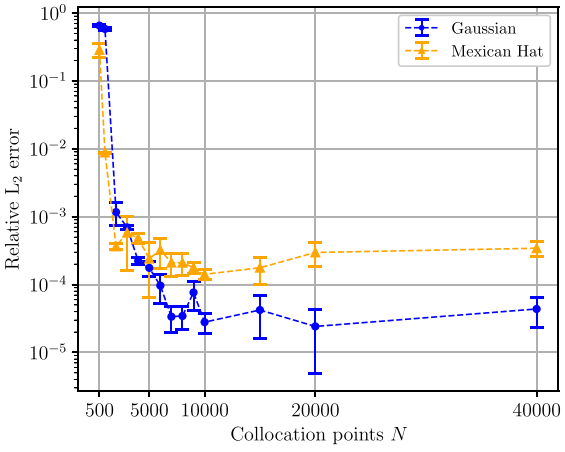

Fig. 6. A plot for relative L_2-error variation with number of collocation points (N) for Gaussian (blue) and Mexican-Hat (orange) wavelets.

In addition, we investigate the relationship between the hyperparameters of W-PINN and its performance for this example. Fig. 5 illustrates the relationship between the resolution level ($[0, J]$) and the relative L_2-error. Here, we employ a network with 8 hidden layers, each containing 100 neurons, and fix 10,000 collocation points. It shows that W-PINN gives optimal results for a specific resolution range (not too high nor too low). Lower resolution is incapable of capturing sharp singularities, and a large J makes the loss function too complex to be optimized. The relatively small error bars demonstrate that W-PINN is numerically stable and well-conditioned with respect to random initialization.

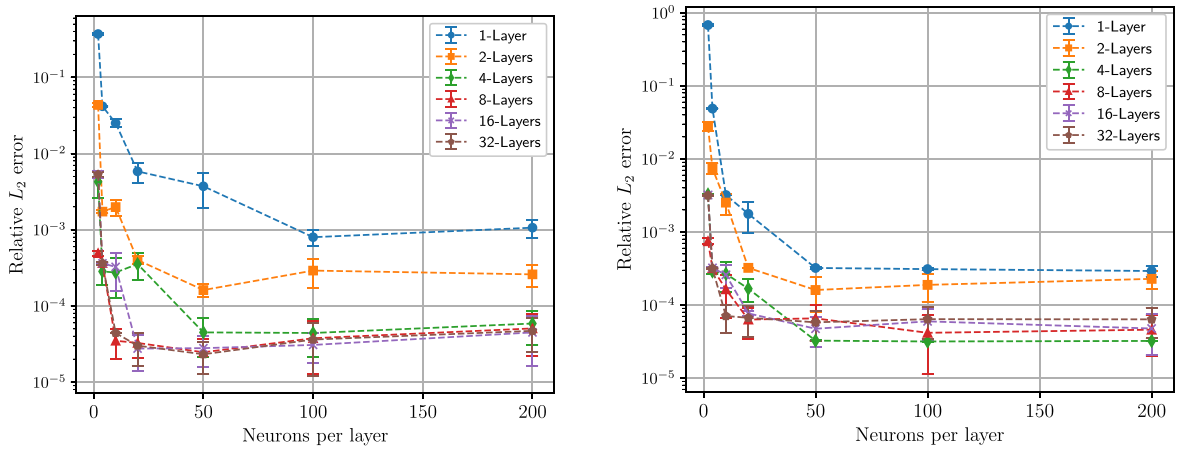

Fig. 7. Impact of number of hidden layers and number of neurons per layer on performance of W-PINN using Gaussian wavelet(left) and Mexican-Hat wavelet(right).

Table 6
Parameters used for solving Example 3.

Parameters	Value
Translation parameter γ	1.0
Set of resolutions (J_M/J_G)	[0,8]/[0,9]
Number of hidden layers	8
Neurons per layer	50
Number of collocation points	10^4
Maximum number of iterations	10^4

Fig. 6 presents the effect of the number of collocation points on performance. We consider the identical network as previously used, with the Gaussian resolutions ($J_G = [0, 12]$) and the Mexican hat resolutions ($J_M = [0, 11]$). It is evident that the relative L_2-error decreases as the number of collocation points increases. However, the error decrease is not significant beyond a certain threshold. After a certain number of collocation points, additional points do not come with any new information about the behavior of the problem under consideration.

Fig. 7 presents experiments on the trainable part of W-PINN. We employ $10,000$ collocation points for both Gaussian and Mexican hat, with resolutions of $[0, 12]$ and $[0, 11]$, respectively. It implies that a shallow network is incapable of learning the behaviors of the problem, regardless of the number of neurons per layer. However, a network with a significant number of layers and sufficient neurons per layer performs adequately. It is redundant to employ a large number of layers and neurons, which increases computational cost rather than accuracy.

Example 3. Consider the following singularly perturbed non-linear problem with Neumann boundary conditions

$$\begin{cases} -\epsilon \dfrac{d^2u}{dt^2} + u^5 + 3u - 1 = 0, t \in [0, 1], \\ u'(0) = \sin(0.5), u'(1) = \exp(-0.7). \end{cases} \tag{11}$$

This example corresponds to a singularly perturbed non-linear problem with Neumann boundary conditions. This problem possesses a boundary layer singularity at both ends. The exact solution to this problem is unknown, so we obtained a numerical solution using Scipy solver and treated it as an exact solution.

Table 6 lists the network parameters used for Example 3. The corresponding numerical results are presented in Table 7 and Fig. 8. Specifically, the Table 7 compares the performance of W-PINN, traditional PINN, and the PINN with a Gaussian wavelet as activation function (Uddin et al., 2023) for various ϵ in terms of relative L_2-error and average training time. This table shows that W-PINN takes much less training time and approximates the solution to a better extent. Fig. 8 demonstrates that as the singularity becomes more prominent, W-PINN effectively addresses the singularity to a greater degree at both ends. In contrast, other PINN methods couldn't resolve the singularity and introduced unnatural oscillations over the domain.

Table 7
Performance comparison of various methods for solving Example 3.

ϵ	Method	L_2-error	Average Training Time
$\epsilon = 2^{-4}$	PINN	8.6e-05	2m 26s
	Wavelet Activation (Uddin et al., 2023)	3.2e-04	4m 49s
	W-PINN (Gaussian)	4.7e-05	28s
	W-PINN (Mexican hat)	1.4e-04	34s
$\epsilon = 2^{-7}$	PINN	1.8e-03	4m 41s
	Wavelet Activation (Uddin et al., 2023)	4.1e-03	6m 13s
	W-PINN (Gaussian)	8.9e-06	14s
	W-PINN (Mexican hat)	8.3e-05	21s
$\epsilon = 2^{-10}$	PINN	5.3e-03	7m 14s
	Wavelet Activation (Uddin et al., 2023)	8.4e-03	12m 28s
	W-PINN (Gaussian)	6.5e-06	13s
	W-PINN (Mexican hat)	9.9e-05	28s

Example 4. FitzHugh-Nagumo (FHN) model (Su, 1993):

The FitzHugh-Nagumo (FHN) model is a simplified version of the Hodgkin-Huxley model, which simulates the dynamics of spiking neurons. It captures how a neuron generates electrical signals (spikes) in response to stimulation (an external current). The mathematical form of the FHN dynamical model is defined as follows:

$$\begin{cases} \dfrac{dv}{dt} - v + \dfrac{v^3}{3} + w - RI = 0, \\ \tau \dfrac{dw}{dt} - v + bw + a = 0, \end{cases} \tag{12}$$

where v denotes the neuron's membrane voltage, w is the recovery variable, reflecting the activation state of ion channels, I represents the external current applied to the neuron, and R is the resistance across the neuron. In addition, a and b are scaling parameters, and τ denotes the time constant for the recovery variable (w), which acts as a singularly perturbed parameter for this model. In particular, for $a = b = 0$, the FHN model describes the Van der Pol oscillator, which describes self-sustaining oscillations in many systems, such as heartbeats, economies, and electronic circuits.

For the numerical computations, we set: $a = 1, b = 1, I = 0.1, R = 1$, and $\tau = 2^{-10}$, $v(0) = 0.5$ and $w(0) = 0.1$. The dynamics of the system are computed till $t = 1$. Table 8 provides the parameters for this Example 4.

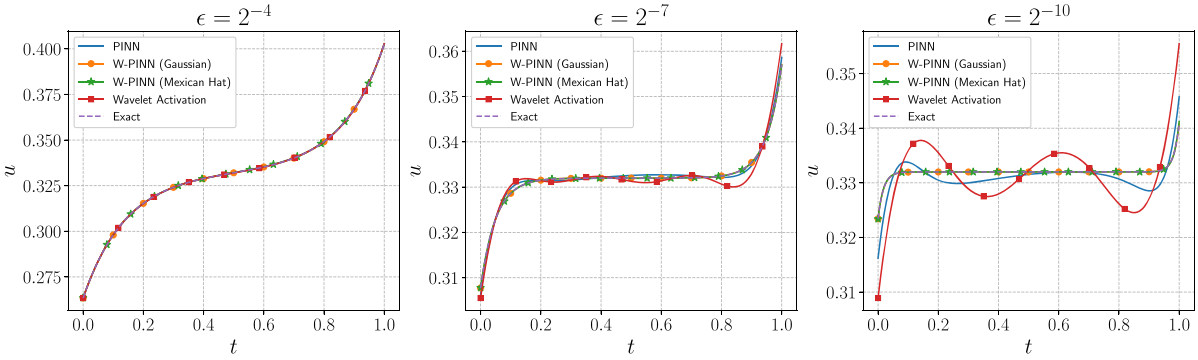

Fig. 8. Comparison of solutions obtained by PINN, PINN with wavelet activation, and W-PINN methods for Example 3. From left to right: with $\epsilon = 2^{-4}$, $\epsilon = 2^{-7}$, and $\epsilon = 2^{-10}$.

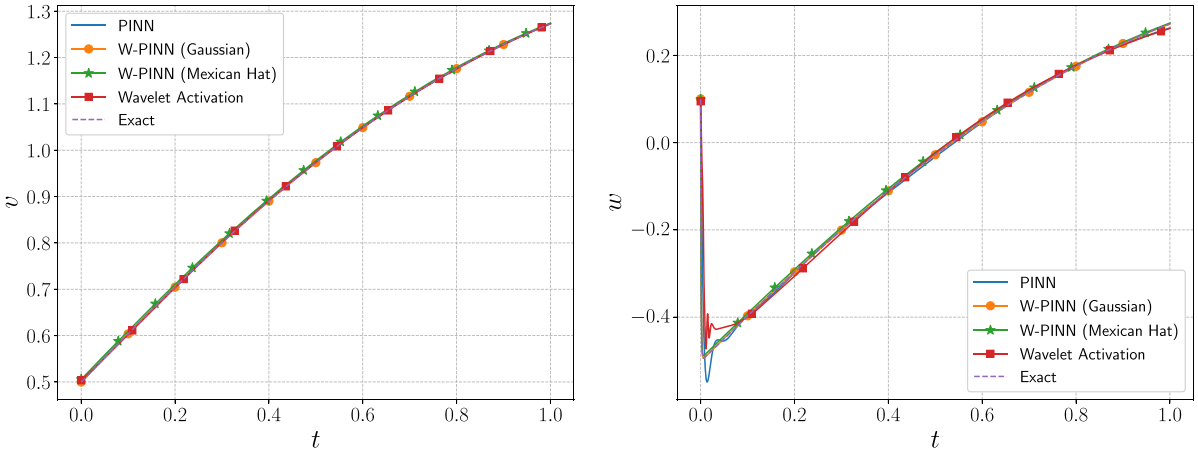

Fig. 9. Comparison of solution of FHN model (4) obtained by PINN, PINN with wavelet activation, and W-PINN methods with $\tau = 2^{-10}$.

Table 8
Parameters used for solving Example 4.

Parameters	Value
Translation parameter γ	1.0
Set of resolutions (J_M/J_G)	[0,10]/[0,11]
Number of hidden layers	10
Neurons per layer	100
Number of collocation points	10^4
Number of iterations	2×10^4

Table 9
Performance comparison of various methods for solving Example 4 with $\tau = 0.15$.

Methods	L_2-error (v/w)	Average Training Time
PINN	$5.9e-04$ $8.1e-02$	6m 38s
Wavelet Activation	$2.1e-03$ 0.11	8m 22s
W-PINN (Gaussian)	$9.2e-05$ $5.7e-04$	1m 06s
W-PINN (Mexican hat)	$4.4e-03$ $8.2e-03$	1m 36s

From Table 9, it is evident that W-PINN gets a more accurate prediction and takes significantly less training time. Fig. 9 compares the W-PINN approximations of v and w with PINN and PINN with wavelet activation. The figure illustrates that a strong initial layer is present in the solution profile of w, which both PINN and PINN with wavelet activation fail to capture and generate spurious oscillations near the singularity, whereas W-PINN effectively resolves the singularity.

Table 10
Parameters used for solving Example 5.

Parameters	Value
Translation hyperparameter γ	0.2
Set of resolutions (J_x, J_t)	[-6,5], [-6,5]
Number of hidden layers	6
Neurons per layer	50
Number of collocation points	10^4
Number of boundary points	10^3
Number of initial points	5×10^2

Table 11
Performance comparison of various methods for solving Example 5.

Methods	L_2-error	Average Training Time
Conventional PINN	$1.07 \pm 0.11 \times 10^0$	-
SA-PINN (McClenny & Braga-Neto, 2023)	$1.76 \pm 0.35 \times 10^{-3}$	66.75 min
PIRBN (Bai et al., 2023)	$7.73 \pm 1.62 \times 10^{-3}$	178.33 min
Gabor PINN (Alkhalifah & Huang, 2024)	$2.01 \pm 2.14 \times 10^{-3}$	42.35 min
MMPINN-DNN (Wang et al., 2024)	$5.01 \pm 1.52 \times 10^{-4}$	11.02 min
W-PINN (proposed method)	$2.56 \pm 1.3 \times 10^{-4}$	4.5 min

Example 5. A heat conduction problem with large gradients (Lan, 2022):

The heat conduction problem investigates temperature flow when an intense heat source suddenly appears. It represents a practical challenge often seen in fusion applications, where researchers need to understand and model rapid heat transfer.

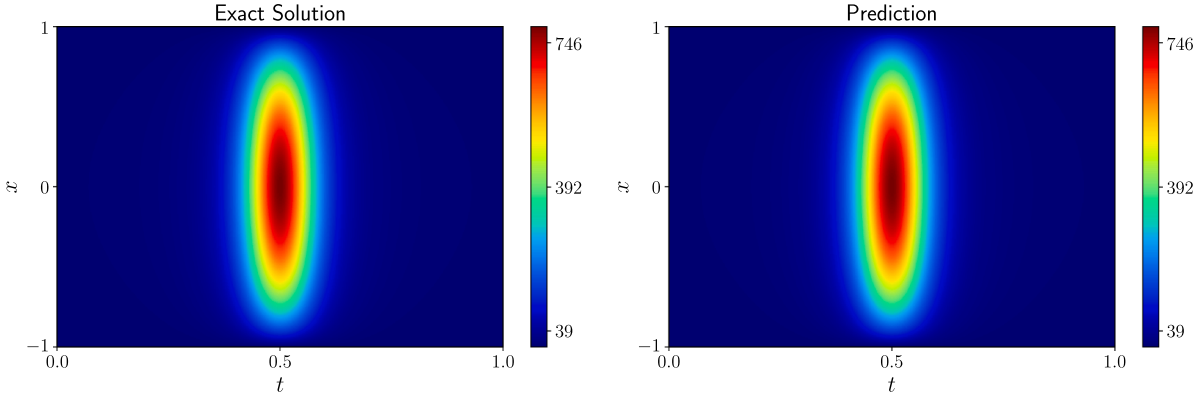

Fig. 10. The exact solution (left) and the prediction of W-PINN (right) with $\epsilon = 0.15$ for high gradient heat conduction problem 5.

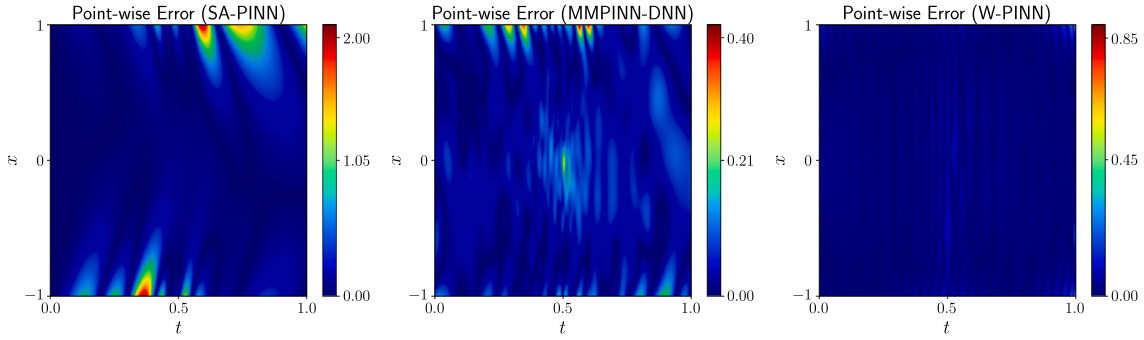

Fig. 11. Point-wise absolute error for Example 5. From left to right: SA-PINN, MMPINN-DNN, and W-PINN.

The following mathematical model includes a small positive constant ϵ that creates steep temperature gradients, making it a useful test case for our developed method:

$$\begin{cases} \dfrac{du}{dt} = \dfrac{d^2u}{dx^2} + f(x,t), x \in (-1,1), t \in [0,1], \\ u(x,0) = (1-x^2)\exp\left(\dfrac{1}{1+\epsilon}\right), x \in (-1,1), \\ u(-1,t) = 0, u(1,t) = 0, t \in (0,1], \end{cases} \tag{13}$$

where f is chosen such that $u(x,t) = (1-x^2)\exp\left(\dfrac{1}{(2t-1)^2+\epsilon}\right)$ is the exact solution.

The behavior of this model exhibits interesting characteristics depending on the value of the parameter ϵ. However, when ϵ becomes very small, the solution shows dramatic changes near $t = 0.5$, exhibiting distinct multiscale characteristics. An important observation is the relationship between the supervised loss term and the residual term - while the boundary conditions ensure the supervised loss term remains minimal (as the boundary value is 0 and the initial value stays below ϵ), the residual term grows considerably as ϵ decreases. This is quantitatively demonstrated as, when $\epsilon = 0.15$ the ratio of $\mathcal{L}_{bc} : \mathcal{L}_{ic} : \mathcal{L}_{res} = 1 : 10 : 10^7$. Traditional PINN methods generally perform well for smooth problems, but face challenges when dealing with such large disparities between supervised and residual terms. Our proposed W-PINN can effectively handle such loss imbalances.

Table 10 represents the parameters of the network for this Example 5. Further, Table 11 presents a comparative analysis of different methods for solving Example 5 with $\epsilon = 0.15$, evaluated in terms of relative L_2-error and average training time. The conventional PINN fails to produce an accurate approximation for this problem. The self-adaptive PINN (SA-PINN)(McClenny & Braga-Neto, 2023) significantly improves accuracy, however, this improvement is accompanied by a substantial increase in training time. Methods such as PIRBN (Bai et al., 2023) and Gabor PINN (Alkhalifah & Huang, 2024) also achieve improved accuracy compared to the conventional PINN, but they require considerably higher training time. The multi-magnitude PINN (MMPINN-DNN)(Wang et al., 2024) attains better accuracy with moderate training time. In contrast, the proposed W-PINN provides the most favorable balance between accuracy and efficiency, achieving the lowest L_2-error among all compared methods while requiring significantly less training time. These results underscore the effectiveness of the wavelet-based formulation in efficiently capturing multiscale features.

Fig. 10 demonstrates the similarity between the exact solution and the W-PINN's prediction for Example 5. Fig. 11 shows point-wise absolute error comparisons across three approaches: SA-PINN, MMPINN-DNN, and W-PINN. This error plot shows that errors for W-PINN throughout the domain remain consistently low except for extremely small regions near corners. Further, Fig. 12 demonstrates the training progress of conventional PINN and W-PINN by showing their loss components over iterations. PINN shows unstable behavior with the boundary and initial condition losses initially increasing, suggesting potential training difficulties. In contrast, W-PINN exhibits a much more stable and efficient training pattern, where all three loss components (residual, initial condition, and boundary condition) consistently decrease from the start, reaching lower magnitudes in fewer iterations. This indicates that W-PINN achieves better optimization performance than the standard PINN approach.

Example 6. Helmholtz equation with high-frequency:

The Helmholtz equation is a crucial elliptic partial differential equation that models electromagnetic wave behavior. It appears in wide applications in physics and engineering. The equation combines the Laplacian operator (Δu) with a wave number term (c^2), and is typically studied on bounded domains with appropriate boundary conditions. In the high-frequency regime, this equation becomes particularly challenging

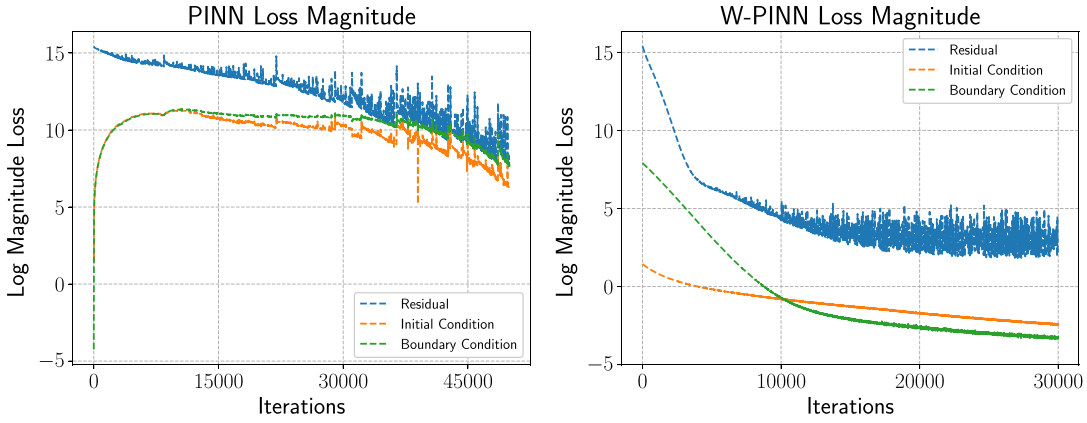

Fig. 12. Loss curve of PINN (left) and W-PINN (right) for Example 5 with $\epsilon = 0.15$.

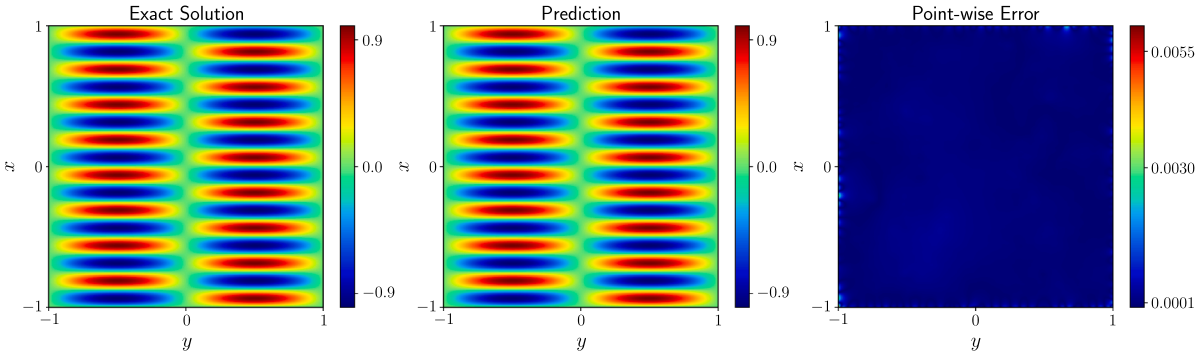

Fig. 13. From left to right: The exact solution, the predicted solution, and the point-wise absolute error using W-PINN for the Helmholtz Eq. 6.

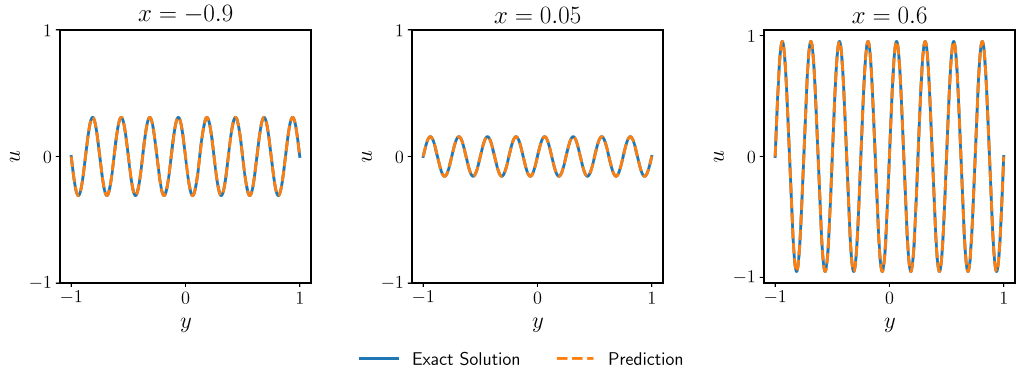

Fig. 14. Comparison of the exact solution (solid blue line) and the prediction (dashed orange line) using W-PINN at different spatial stamps for Example 6.

Table 12
Parameters used for solving Example 6.

Parameters	Value
Translation hyperparameter γ	0.2
Set of resolutions (J_x, J_y)	[-4,5], [-4,5]
Number of hidden layers	6
Neurons per layer	50
Number of collocation points	10^4
Number of boundary points	10^3

Table 13
Performance comparison of various methods for solving Example 6.

Methods	L_2-error	Average Training Time
Conventional PINN	4.93 $\pm$1.56 $\times 10^{-2}$	23.18 min
SA-PINN (McClenny & Braga-Neto, 2023)	1.27$\pm$1.11$\times$ 10^{-2}	152.88 min
PIRBN(Bai et al., 2023)	8.82 $\pm$1.32 $\times 10^{-3}$	38.32 min
Gabor PINN(Alkhalifah & Huang, 2024)	3.93 $\pm$2.07 $\times 10^{-3}$	24.17 min
MMPINN-MFF(Wang et al., 2024)	6.56 $\pm$4.17 $\times 10^{-4}$	22.75 min
MMPINN-INN(Wang et al., 2024)	2.13 $\pm$0.43 $\times 10^{-4}$	64.29 min
W-PINN (proposed method)	3.12 $\pm$0.71$\times 10^{-4}$	6.3 min

to solve numerically due to the oscillatory nature of its solutions.

$$\begin{cases} \Delta u(x,y) + c^2 u(x,y) = f(x,y), & (x,y) \in \Omega = (-1,1)\times(-1,1), \\ u(x,y) = p(x,y), & (x,y) \in \partial\Omega. \end{cases} \tag{14}$$

f and p are determined in such a way $u(x,y) = \sin(b_1\pi x)\sin(b_2\pi y)$ is the exact solution.

Due to the significant initial imbalance between residual and supervised components, conventional PINN fails to provide accurate predictions as the optimization process becomes heavily skewed towards residual minimization (Wang et al., 2021a).

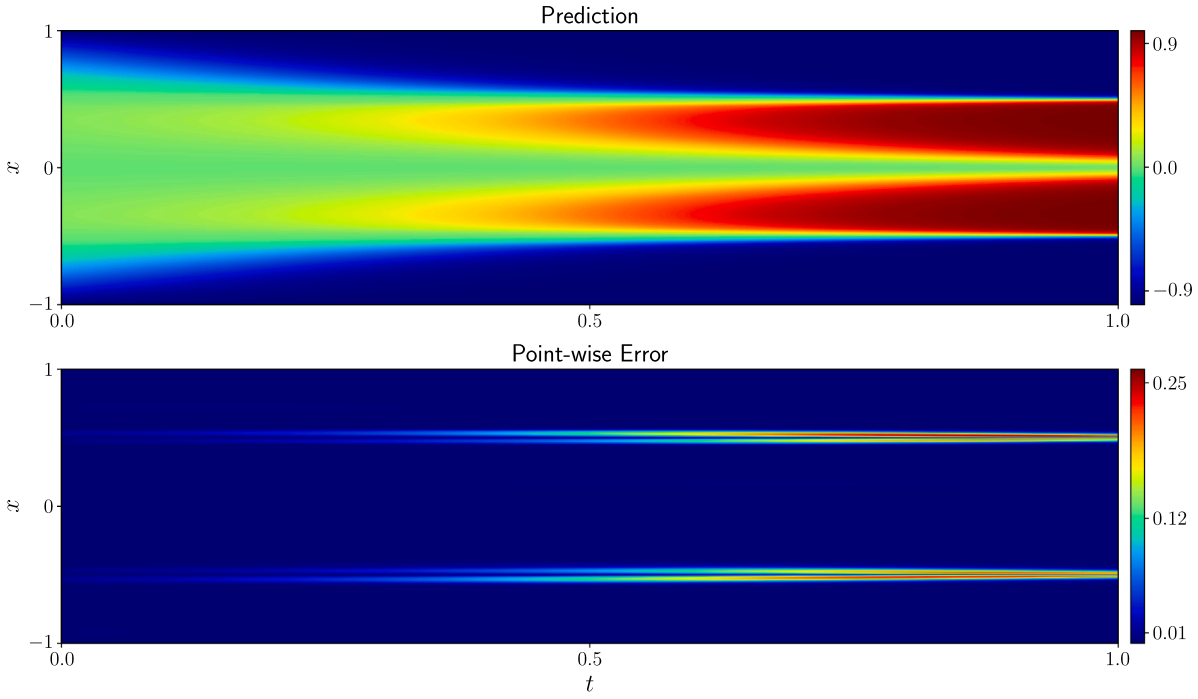

Fig. 15. Top: Predicted solution of Allen-Cahn Eq. 7 via W-PINN. Bottom: point-wise absolute error distribution.

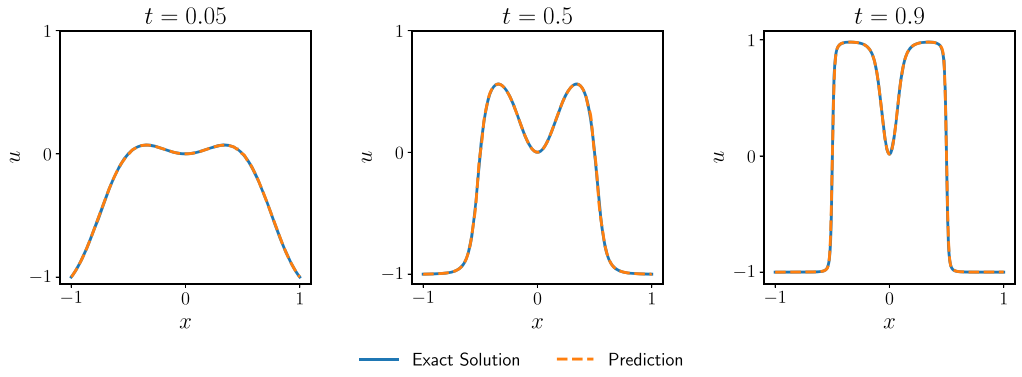

Fig. 16. Comparison of exact solution (solid blue line) and predicted solution (dashed orange line) using W-PINN at different time instances for Example 7.

Table 14
Parameters used for solving Example 7.

Parameters	Value
Translation parameter γ	0.4
Resolutions (J_x, J_t)	[-5,6], [-5,5]
Number of hidden layers	6
Neurons per layer	100
Number of collocation points	2×10^4
Number of boundary points	2×10^3
Number of initial points	10^3

Table 15
Performance comparison of various methods for solving Example 7 with $\epsilon = 10^{-4}$.

Methods	L_2-error
Conventional PINN	0.96 ± 0.06
Time-adaptive approach (Wight & Zhao, 2021)	$8.0 \times 10^{-2} \pm 0.56 \times 10^{-2}$
SA-PINN (McClenny & Braga-Neto, 2023)	$2.1 \times 10^{-2} \pm 1.21 \times 10^{-2}$
W-PINN (proposed method)	$4.8 \times 10^{-2} \pm 0.6 \times 10^{-2}$

To test the performance of W-PINN, we set the model parameters as $c = 1, b_1 = 1$, and $b_2 = 8$. Table 12 represents the parameters of the network for this Example 6. The comparative analysis presented in Table 13 demonstrates that W-PINN achieves comparable or superior accuracy in terms of L_2-error relative to state-of-the-art methods in recent literature. Along with its significantly reduced training time, W-PINN is established as a particularly attractive framework. Fig. 13 illustrates the remarkable agreement between the analytical solution and W-PINN predictions across the entire domain. In Fig. 14, cross-sectional comparisons at various x-coordinates further validate the method's accuracy, with predicted solutions indistinguishable from the exact solutions.

Table 16
Parameters used for solving Example 8.

Parameters	Value
Translation hyperparameter γ	0.5
Resolutions (J_x, J_t)	[-5,5], [-5,5]
Number of hidden layers	8
Neurons per layer	50
Number of collocation points	10^4
Number of boundary points	10^3
Number of initial points	5×10^2

Example 7. Allen-Cahn reaction-diffusion equation:

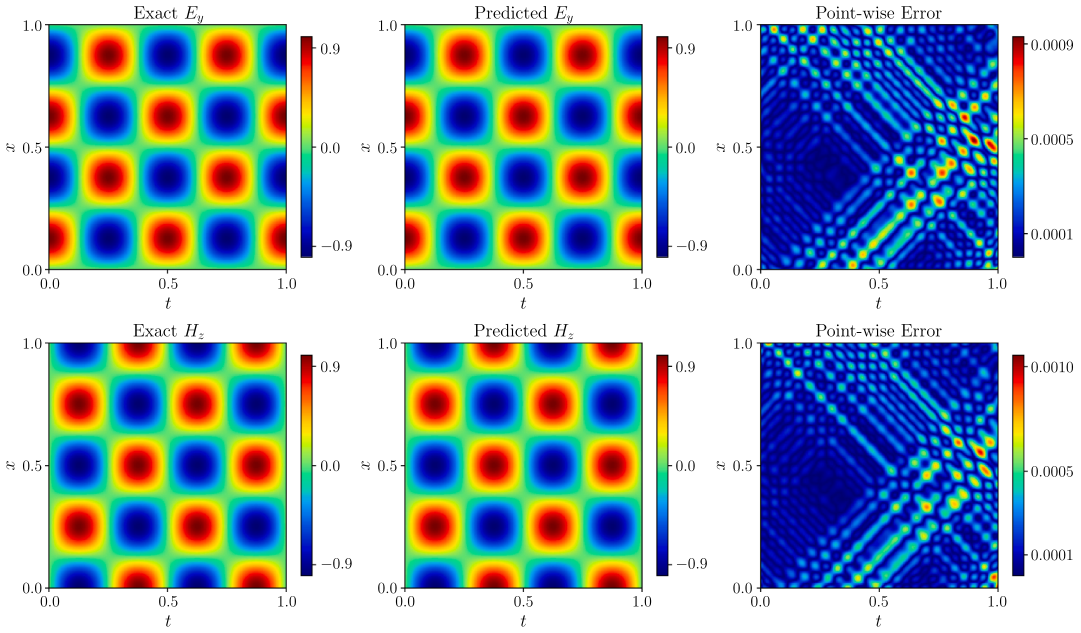

Fig. 17. Solution of Maxwell's Eq. 17 in homogeneous medium using W-PINN. Top: E_y Exact, predicted, and corresponding point-wise absolute error. Bottom: H_z Exact, predicted, and corresponding point-wise absolute error.

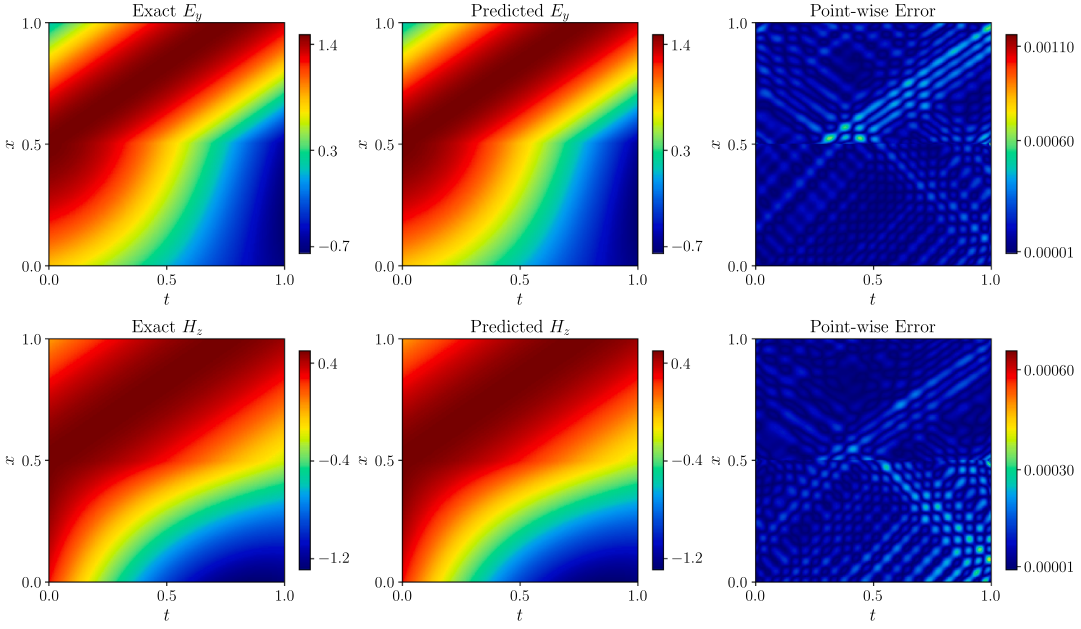

Fig. 18. Solution of Maxwell's Eq. 17 in heterogeneous medium using W-PINN. Top: E_y Exact, predicted, and corresponding point-wise absolute error. Bottom: H_z Exact, predicted, and corresponding point-wise absolute error.

Table 17
Parameters used for solving Example 9.

Parameters	Value
Translation hyperparameter γ	0.5
Resolutions (J_x, J_y)	[-15,5], [-15,5]
Number of hidden layers	12
Neurons per layer	100
Number of collocation points	4×10^4
Number of boundary points	2×10^3

The Allen-Cahn reaction-diffusion PDE is a widely used model in materials science, particularly for simulating phase separation processes in metallic alloys (Kunselman et al., 2020; Moelans et al., 2008). The equation describes the evolution of an order parameter, u, which represents the phase state of the material. The PDE combines a diffusion term with a non-linear reaction term that drives the phase separation. The Allen-Cahn equation used in this study is defined as:

$$\begin{cases} u_t - \epsilon u_{xx} + 5u^3 - 5u = 0, \quad x \in [-1, 1], \quad t \in [0, 1], \\ u(t, -1) = u(t, 1), \\ u_x(t, -1) = u_x(t, 1), \\ u(x, 0) = x^2 \cos(\pi x), \end{cases} \tag{15}$$

here ϵ denotes a singularly perturbed parameter. For numerical computations, we choose $\epsilon = 10^{-4}$. Moreover, the exact solution to this

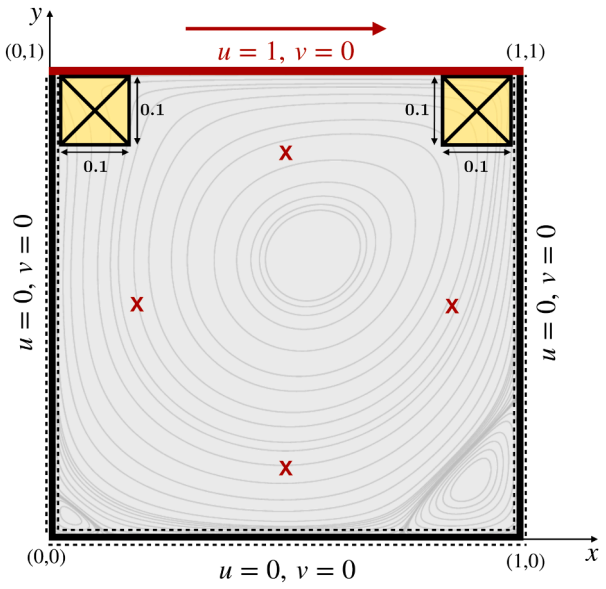

Fig. 19. A lid-driven unit square cavity. For $Re = 400$: additional collocation points are sampled from the yellow crossed region, and red crosses indicate reference points.

problem is unknown, so we obtained a numerical solution using Scipy solver and treated it as the exact solution. The Allen-Cahn equation is particularly challenging due to its rapid changes across space and time, making it difficult to model accurately. Additionally, its periodic boundary conditions provide an extra layer of complexity, making it an ideal benchmark for testing the model's capabilities.

Table 14 provides the parameters used for this problem. Table 15 presents a comparative analysis of different PINN-based methods. The conventional PINN completely fails to capture the complex dynamics of the problem. While the time-adaptive method (Wight & Zhao, 2021) shows some improvement, its accuracy remains unsatisfactory. In contrast, our proposed W-PINN achieves accuracy comparable to SA-PINN (McClenny & Braga-Neto, 2023) while demonstrating computational efficiency with approximately twice the speed (30 ms/iteration) of SA-PINN. Fig. 15 illustrates the predicted dynamics using the W-PINN method, where the predominance of dark blue regions in the error distribution plot indicates consistently low prediction errors across the solution domain. Further, Fig. 16 compares W-PINN prediction with the exact solution at three different time stamps, validating the model's ability to capture the temporal evolution of the system dynamics accurately.

Example 8. Maxwell's Equation:

Maxwell's equations represent the fundamental principles governing electromagnetic theory, describing the relationships between electric and magnetic fields and their interaction with matter. Their applications span numerous fields of engineering and physics. In telecommunications, these equations govern the propagation of electromagnetic waves through optical fibers and wireless channels. The medical industry relies heavily on Maxwell's equations for diagnostic imaging technologies, particularly in magnetic resonance imaging (MRI) (Brown et al., 2014). In the aerospace sector, these equations are essential for radar system design and satellite communication systems. In their differential form, Maxwell's equations are expressed as:

$$\begin{cases} \nabla \times \boldsymbol{E}(t,\boldsymbol{x}) &= -\mu(\boldsymbol{x})\dfrac{\partial \boldsymbol{H}(t,\boldsymbol{x})}{\partial t}, \\ \nabla \times \boldsymbol{H}(t,\boldsymbol{x}) &= \epsilon(\boldsymbol{x})\dfrac{\partial \boldsymbol{E}(t,\boldsymbol{x})}{\partial t}, \end{cases} \tag{16}$$

where $\boldsymbol{E}$ is the electric field vector, $\boldsymbol{H}$ is the magnetic field vector. The material properties are characterized by μ, the magnetic permeability, which quantifies the medium's response to magnetic fields, and ϵ, the electric permittivity, which describes the medium's capacity to store electrical energy. The operators ∇ and $\times$ represent the gradient operator and cross product, respectively.

A significant challenge in solving these equations arises from their inherent rapid oscillations in both space and time. This characteristic poses particular difficulties for traditional PINNs, especially when dealing with heterogeneous media where μ and ϵ exhibit discontinuous behavior at media interfaces. Here, we present a solution of Maxwell's equation in both a homogeneous and a heterogeneous medium via W-PINN and compare them with the traditional PINN.

A. Homogeneous media

We first investigate the performance of our proposed method by solving Maxwell's equations in a one-dimensional cavity model with homogeneous media. The governing equations are:

$$\frac{\partial E_y}{\partial t} = -\frac{1}{\epsilon}\frac{\partial H_z}{\partial x}, \frac{\partial H_z}{\partial t} = -\frac{1}{\mu}\frac{\partial E_y}{\partial x}, \quad x \in [0,1], \quad t \in [0,1]. \tag{17}$$

The system is subject to perfectly electric conductor (PEC) boundary conditions:

$$\begin{cases} E_y(0,t) = E_y(1,t) = 0, \\ \dfrac{\partial H_z(x,t)}{\partial x}\Big|_{x=0,1} = 0. \end{cases} \tag{18}$$

The ground truth for the example is given by:

$$\begin{cases} E_y = \sin(n\pi x)\cos(\omega t), \\ H_z = -\cos(n\pi x)\sin(\omega t), \end{cases} \tag{19}$$

where $n = 4$ is the order of cavity mode, cavity frequency $\omega = n\pi/l$ and l is the length of medium.

Over five random runs, W-PINN achieved average relative L_2-errors of $5.47 \pm 1.12 \times 10^{-4}$ and $5.51 \pm 2.01 \times 10^{-4}$ for E_y and H_z, respectively. In contrast, traditional PINNs exhibited notably higher errors of $3.08 \pm 1.18 \times 10^{-3}$ and $5.18 \pm 1.93 \times 10^{-3}$. The computational efficiency of W-PINN is evident in its training time, which averages 6.2 minutes over roughly 2×10^4 iterations on an Nvidia A6000 GPU. The network architecture parameters are tabulated in Table 16. Fig. 17 presents the W-PINN predictions for both electric and magnetic fields, alongside their corresponding point-wise errors within the cavity. The remarkably low point-wise error distribution validates the high accuracy of our approach in capturing the electromagnetic field behavior.

B. Heterogeneous media

For this case, we chose Maxwell's Eq. 17 in a heterogeneous medium with the following analytical solution:

$$E_y = \begin{cases} \cos(2t - 2x + 1) \\ +0.5\cos(2t + 2x - 1), & \text{if } x \in \Omega_1, \\ 1.5\cos(2t - 3x + 1.5), & \text{if } x \in \Omega_2, \end{cases} \tag{20}$$

$$H_z = \begin{cases} \cos(2t - 2x + 1) \\ -0.5\cos(2t + 2x - 1), & \text{if } x \in \Omega_1, \\ 0.5\cos(2t - 3x + 1.5), & \text{if } x \in \Omega_2, \end{cases} \tag{21}$$

where $\Omega_1 = [0, 0.5]$ and $\Omega_2 = [0.5, 1]$. Here $\mu = 1, \epsilon = 1$ in Ω_1 and $\mu = 4.5, \epsilon = 0.5$ in Ω_2. According to the electromagnetic interface condition:

$$\begin{cases} E_y(0.5,t)\big|_{x\in\Omega 1} = E_y(0.5,t)\big|_{x\in\Omega 2}, \\ H_z(0.5,t)\big|_{x\in\Omega 1} = H_z(0.5,t)\big|_{x\in\Omega 2}. \end{cases} \tag{22}$$

In this challenging context of heterogeneous media, W-PINN demonstrates remarkable accuracy, achieving average relative L_2-errors of

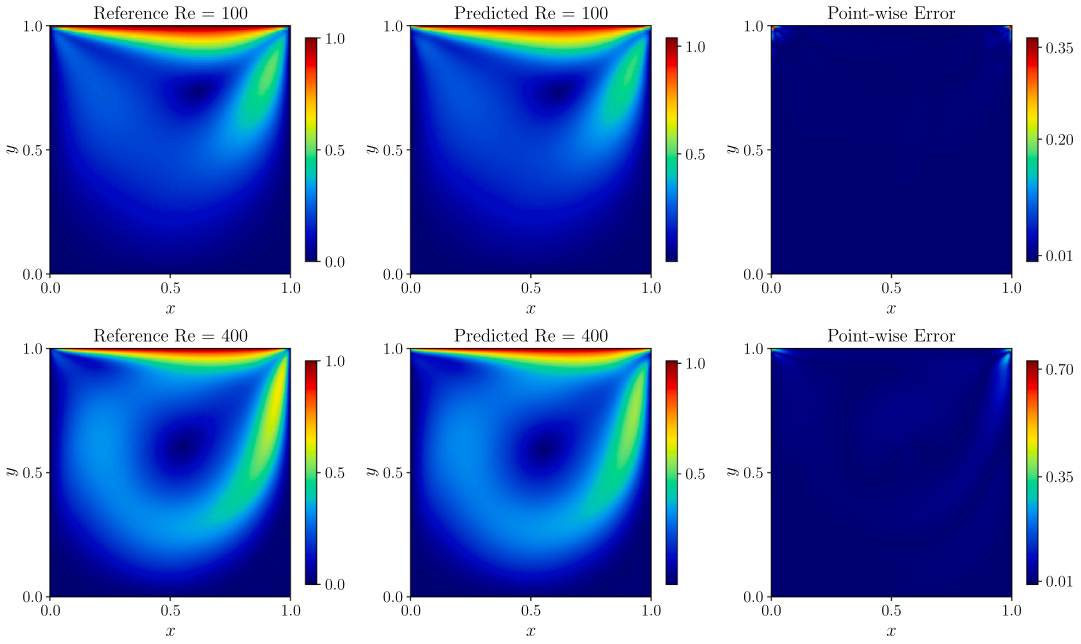

Fig. 20. From left to right: The reference fluid flow, the W-PINN predicted fluid speed, and the point-wise absolute error for the steady-state lid-driven cavity flow (23) for Re = 100 (top) and Re = 400 (bottom).

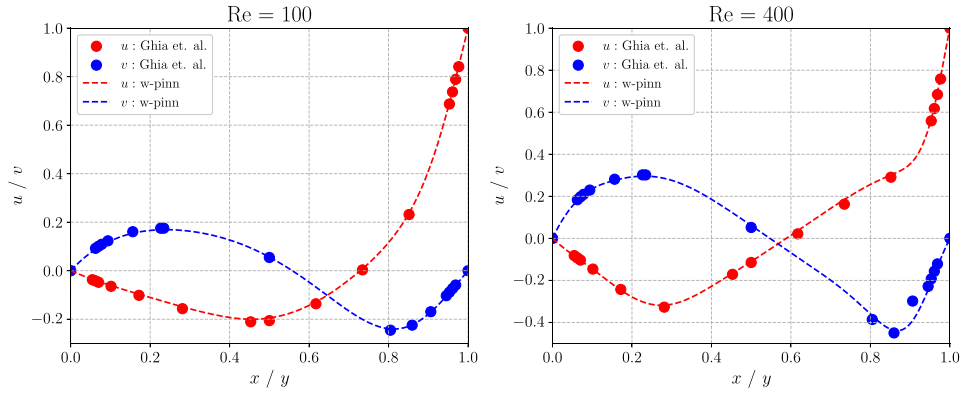

Fig. 21. Comparison of W-PINN solution (dashed line) with Ghia's benchmark (scattered circles) for the steady-state lid-driven cavity flow (23) for Re = 100 (left) and Re = 400 (right). These data points lie on the horizontal and vertical lines through the geometric center of the cavity, for red color: $x = 0.5$ and blue color: $y = 0.5$.

$2.41 \pm 1.22 \times 10^{-4}$ and $2.81 \pm 1.51 \times 10^{-4}$ for electric (E_y) and magnetic (H_z) fields, respectively. These results represent a two-order-of-magnitude improvement over traditional PINNs, which exhibit substantially higher errors of $1.37 \pm 1.18 \times 10^{-2}$ and $2.07 \pm 0.51 \times 10^{-2}$. W-PINN took an average of 16.1 minutes for about 3×10^4 iterations on the Nvidia A6000 GPU. The network parameters are the same as in Table 16. Fig. 18 illustrates the W-PINN predictions for both electric and magnetic fields, alongside their corresponding point-wise absolute errors. While the media interface presents the greatest computational challenge, W-PINN maintains impressive accuracy throughout the domain, successfully capturing the Electromagnetic field behavior across the media.

Example 9. Lid-Driven Cavity flow:

The steady state two-dimensional incompressible lid-driven cavity flow is a classic benchmark problem in computational fluid dynamics. The flow is governed by the incompressible Navier-Stokes equations, which present significant challenges due to their non-linear nature and the presence of pressure-velocity coupling. As illustrated in Fig. 19, the problem consists of a unit square cavity where the top wall moves with a uniform velocity of 1 unit while maintaining no-slip conditions on all other walls. The governing Navier-Stokes equations for this system are:

$$\begin{cases} \dfrac{\partial u}{\partial x} + \dfrac{\partial v}{\partial y} = 0, \\ u\dfrac{\partial u}{\partial x} + v\dfrac{\partial u}{\partial y} + \dfrac{\partial p}{\partial x} - \dfrac{1}{\mathrm{Re}}\left(\dfrac{\partial^2 u}{\partial x^2} + \dfrac{\partial^2 u}{\partial y^2}\right) = 0, \\ u\dfrac{\partial v}{\partial x} + v\dfrac{\partial v}{\partial y} + \dfrac{\partial p}{\partial y} - \dfrac{1}{\mathrm{Re}}\left(\dfrac{\partial^2 v}{\partial x^2} + \dfrac{\partial^2 v}{\partial y^2}\right) = 0, \end{cases} \tag{23}$$

where u and v are the fluid velocities in x and y directions respectively, and p is the fluid pressure. Re is the Reynolds number, which represents the ratio of inertial forces to viscous forces. We evaluate our W-PINN approach for two distinct flow regimes: $Re = 100$ and $Re = 400$, and compare the results with the solution obtained through COMSOL and also validate against the widely accepted benchmark solutions of Ghia et al. (1982).

For this example, the network's parameters are tabulated in Table 17. Fig. 20 presents the predicted fluid flow speed distribution ($|V| = \sqrt{u^2 + v^2}$), along with a comparison with the COMSOL reference solution, demonstrating successful capture of the characteristic

vortex structure. The relative L_2-error for $Re = 100$ and $Re = 400$ are $2.75 \pm 1.02 \times 10^{-2}$ and $6.17 \pm 1.05 \times 10^{-2}$ respectively. W-PINN took about 20 minutes of training time for 10^4 iterations on Nvidia A6000. At the higher Reynolds number of 400, where flow complexity increases, we uniformly sampled an additional 10^4 collocation points from the upper left and upper right corners of the cavity (crossed region in Fig. 19). In addition, only four strategically chosen reference points (marked by red crosses in Fig. 19) were added. Interestingly, these minimal additions reduced the relative L_2-error roughly by half. A validation against Ghia's benchmark data is illustrated in Fig. 21. These results demonstrate W-PINN's robust capability to handle complex fluid dynamics problems to a good extent.

4. Conclusion

We presented a wavelet basis function representation of PINN (W-PINN), and successfully demonstrated the advantages of W-PINNs over other PINN-based methods in addressing multiscale problems characterized by steep gradients, rapid oscillations, or singular behavior. The wavelet theory served as the foundation for the development of W-PINN, which combined the learning efficiency of PINN with the localization properties of wavelets (both in scale and space) to capture non-linear information within localized regions. A key advantage of the proposed framework is its elimination of the need for automatic differentiation (autograd) in computing residual derivatives, which substantially accelerates training compared to methods that rely on autograd. To support faster convergence behavior, we provide a comparative analysis of W-PINN using NTK theory.

Beyond the problems considered in this work, several promising directions emerge for future research. W-PINN can be naturally extended to parametric PDEs, as well as to inverse problems involving the identification of unknown source terms or governing parameters. The wavelet representation also enables the development of adaptive refinement strategies, in which the resolution level is dynamically increased in regions exhibiting sharp gradients or localized features. A limitation of the proposed method is its dependence on a careful selection of the wavelet basis for optimal performance. Another limitation is the exponential growth in the number of wavelet basis functions with increasing dimensionality, which leads to significant memory overhead and practical challenges in handling large wavelet matrices. Addressing these limitations through sparse representations, dimensionality reduction, or adaptive basis selection constitutes an important direction for future work. Nevertheless, the challenges targeted in this manuscript, namely, multiscale behavior, stiffness, and loss imbalance, are already sufficiently severe that conventional and many advanced PINN variants struggle to solve even in low-dimensional settings. Furthermore, W-PINNs show strong potential for application to other classes of problems that exhibit inherent singular behavior, such as fractional differential equations. These models often present nonlocality and memory effects, where the localized resolution capabilities of wavelets can offer a significant advantage.

Overall, the proposed W-PINN framework provides a robust and efficient alternative to existing physics-informed learning approaches for handling problems exhibiting multiscale behavior, and opens new possibilities for multi-scale representation in scientific machine learning.

CRediT authorship contribution statement

Himanshu Pandey: Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Formal analysis, Data curation, Conceptualization; **Anshima Singh:** Writing – review & editing, Visualization, Validation, Investigation, Formal analysis; **Ratikanta Behera:** Writing – review & editing, Visualization, Validation, Supervision, Methodology, Conceptualization.

Preprint

A preprint of this paper is available on arxiv with Ref. No. arXiv:2409.11847.

Data availability

The data and source codes of problems addressed in this study are available at https://github.com/himanshup21/W-PINN.git.

Declaration of competing interest

The authors declare that there are no conflicts of interest.

Acknowledgments

Ratikanta Behera is supported by the Anusandhan National Research Foundation (ANRF), Government of India, under Grant No. EEQ/2022/001065. We want to express our gratitude to the editor for taking the time to handle the manuscript and to the anonymous referees for their constructive comments.

Appendix A. Hyperparameters

Table A.18
Neural network and optimization hyperparameters for W-PINN.

Problems	Depth shallow + deep	Width neurons per layers	Activation	Optimizer	Max iters
Example 1: 1D advection-diffusion	2 + 4	50	tanh	Adam (lr: 10^{-5})	5×10^4
Example 2: Singularly perturbed nonlinear IVP	2 + 4	50	tanh	Adam (lr: 10^{-5})	10^5
Example 3 Singularly perturbed nonlinear BVP	2 + 6	50	tanh	Adam (lr: 10^{-4})	10^4
Example 4 FHN model	3 + 7	100	tanh	Adam (lr: 10^{-5})	2×10^4
Example 5 Heat conduction with large gradients	2 + 7	50	tanh	Adam (lr: 10^{-4})	2×10^4
Example 6 2D Helmholtz with high-frequency	2 + 4	50	tanh	Adam (lr: 10^{-5})	5×10^4
Example 7 Allen–Cahn with periodic BCs	2 + 4	100	tanh	Adam (lr: 10^{-5})	10^5
Example 8 Maxwell's Equation	2 + 6	50	tanh	Adam (lr: 10^{-4})	3×10^4
Example 9 Lid-driven cavity	3 + 9	100	tanh	Adam (lr: 10^{-5})	10^4

Table A.19
Resolution and sampling parameters for W-PINN.

Problems	Resolution(J)	δ	N_r	N_{bc}	N_{ic}
Example 1 1D advection-diffusion	$J_M, J_G, J_{\mathcal{M}} = [0, 8], [0, 10], [0, 9]$	1.0	10^3	–	–
Example 2 Singularly perturbed nonlinear IVP	$J_M, J_G, J_{\mathcal{M}} = [0, 9], [0, 10], [0, 10]$	1.0	10^4	–	–
Example 3 Singularly perturbed nonlinear BVP	$J_M, J_G = [0, 8], [0, 9]$	1.0	10^4	–	–
Example 4 FHN model	$J_M, J_G = [0, 10], [0, 11]$	1.0	10^4	–	–
Example 5 Heat conduction with large gradients	$J_x = J_t = [-6, 5]$	0.2	10^4	10^3	5×10^2
Example 6 2D Helmholtz with high-frequency	$J_x = J_y = [-4, 5]$	0.2	10^4	10^3	–
Example 7 Allen–Cahn with periodic BCs	$J_x = [-5, 6],\ J_t = [-5, 5]$	0.4	2×10^4	2×10^3	10^3
Example 8 Maxwell's Equation	$J_x = J_t = [-5, 5]$	0.5	10^4	10^3	5×10^2
Example 9 Lid-driven cavity	$J_x = J_y = [-15, 5]$	0.5	4×10^4	2×10^3	–

References

Abedi, M. M., Pardo, D., & Alkhalifah, T. (2026). Gabor-enhanced physics-informed neural networks for fast simulations of acoustic wavefields. *Neural Networks: The Official Journal of the International Neural Network Society*, *193*, 107978. https://doi.org/10.1016/j.neunet.2025.107978

Alkhalifah, T., & Huang, X. (2024). Physics-informed neural wavefields with gabor basis functions. *Neural Networks : The Official Journal of the International Neural Network Society*, *175*, 106286. https://doi.org/10.1016/j.neunet.2024.106286

Arzani, A., Cassel, K. W., & D'Souza, R. M. (2023). Theory-guided physics-informed neural networks for boundary layer problems with singular perturbation. *Journal of Computational Physics*, *473*, Paper No. 111768, 15. https://doi.org/10.1016/j.jcp.2022.111768

Bai, J., Liu, G.-R., Gupta, A., Alzubaidi, L., Feng, X.-Q., & Gu, Y. (2023). Physics-informed radial basis network (PIRBN): A local approximating neural network for solving nonlinear partial differential equations. *Computer Methods in Applied Mechanics and Engineering*, *415*, 116290. https://doi.org/10.1016/j.cma.2023.116290

Baydin, A. l. m. G., Pearlmutter, B. A., Radul, A. A., & Siskind, J. M. (2017). Automatic differentiation in machine learning: A survey. *Journal of Machine Learning Research : JMLR*, *18*, Paper No. 153, 43. http://jmlr.org/papers/v18/17-468.HTML.

Bender, C. M., & Orszag, S. A. (1999). Advanced mathematical methods for scientists and engineers. I. Springer-Verlag, New York. Asymptotic methods and perturbation theory, Reprint of the 1978 original. https://doi.org/10.1007/978-1-4757-3069-2

Brown, R. W., Cheng, Y.-C. N., Haacke, E. M., Thompson, M. R., & Venkatesan, R. (2014). Magnetic resonance imaging: physical principles and sequence design. John Wiley & Sons. https://doi.org/10.1002/9781118633953

Cao, Y., Fang, Z., Wu, Y., Zhou, D.-X., & Gu, Q. (2021). Towards understanding the spectral bias of deep learning. In Z.-H. Zhou (Ed.), *Proceedings of the thirtieth international joint conference on artificial intelligence, IJCAI-21* (pp. 2205–2211). International Joint Conferences on Artificial Intelligence Organization. Main Track. https://doi.org/10.24963/ijcai.2021/304

Cen, Z., Erdogan, F., & Xu, A. (2014). An almost second order uniformly convergent scheme for a singularly perturbed initial value problem. *Numerical Algorithms*, *67*(2), 457–476. https://doi.org/10.1007/s11075-013-9801-0

Cuomo, S., Schiano Di Cola, V., Giampaolo, F., Rozza, G., Raissi, M., & Piccialli, F. (2022). Scientific machine learning through physics-informed neural networks: Where we are and what's next. *Journal of Scientific Computing*, *92*(3), Paper No. 88, 62. https://doi.org/10.1007/s10915-022-01939-z

Cybenko, G. (1989). Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals, and Systems*, *2*(4), 303–314. https://doi.org/10.1007/BF02551274

Daubechies, I. (1992). Ten lectures on wavelets (vol. 61). CBMS-NSF Regional Conference Series in Applied Mathematics. Society for Industrial and Applied Mathematics (SIAM), Philadelphia, PA. https://doi.org/10.1137/1.9781611970104

Farhani, G., Dashtbayaz, N. H., Kazachek, A., & Wang, B. (2025). A simple remedy for failure modes in physics informed neural networks. *Neural Networks : The Official Journal of the International Neural Network Society*, *183*, 106963. https://doi.org/10.1016/j.neunet.2024.106963

Ghia, U., Ghia, K. N., & Shin, C. T. (1982). High-re solutions for incompressible flow using the navier-stokes equations and a multigrid method. *Journal of Computational Physics*, *48*(3), 387–411. https://doi.org/10.1016/0021-9991(82)90058-4

Glorot, X., & Bengio, Y. (2010). Understanding the difficulty of training deep feedforward neural networks. In Y. W. Teh, & M. Titterington (Eds.), *Proceedings of the thirteenth international conference on artificial intelligence and statistics* (pp. 249–256). Chia Laguna Resort, Sardinia, Italy: PMLR (*vol. 9*). Proceedings of Machine Learning Research. https://proceedings.mlr.press/v9/glorot10a.HTML.

Hu, Z., Shukla, K., Karniadakis, G. E., & Kawaguchi, K. (2024). Tackling the curse of dimensionality with physics-informed neural networks. *Neural Networks : The Official Journal of the International Neural Network Society*, *176*, 106369. https://doi.org/10.1016/j.neunet.2024.106369

Huang, X., & Alkhalifah, T. (2022). Single reference frequency loss for multifrequency wavefield representation using physics-informed neural networks. *IEEE Geoscience and Remote Sensing Letters*, *19*, 1–5.

Jacot, A., Gabriel, F., & Hongler, C. (2018). Neural tangent kernel: Convergence and generalization in neural networks. In S. Bengio, H. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi, & R. Garnett (Eds.), *Advances in neural information processing systems*. Curran Associates, Inc. (*vol. 31*). https://proceedings.neurips.cc/paper_files/paper/2018/file/5a4be1fa34e62bb8a6ec6b91d2462f5a-Paper.pdf.

Jagtap, A. D., & Karniadakis, G. E. (2020). Extended physics-informed neural networks (XPINNs): a generalized space-time domain decomposition based deep learning framework for nonlinear partial differential equations. *Communications in Computational Physics*, *28*(5), 2002–2041. https://doi.org/10.4208/cicp.oa-2020-0164

Karniadakis, G. E., Kevrekidis, I. G., Lu, L., Perdikaris, P., Wang, S., & Yang, L. (2021). Physics-informed machine learning. *Nature Reviews Physics*, *3*(6), 422–440. https://doi.org/10.1038/s42254-021-00314-5

Kharazmi, E., Zhang, Z., & Karniadakis, G. E. M. (2021). Hp-VPINNs: Variational physics-informed neural networks with domain decomposition. *Computer Methods in Applied Mechanics and Engineering*, *374*, 113547. https://doi.org/10.1016/j.cma.2020.113547

Kingma, D. P., & Ba, J. (2014). Adam: A method for stochastic optimization. arXiv preprint arXiv:1412.6980, . https://doi.org/10.48550/arXiv.1412.6980

Kunselman, C., Attari, V., McClenny, L., Braga-Neto, U., & Arroyave, R. (2020). Semi-supervised learning approaches to class assignment in ambiguous microstructures. *Acta Materialia*, *188*, 49–62. https://doi.org/10.1016/j.actamat.2020.01.046

Lagaris, I. E., Likas, A., & Fotiadis, D. I. (1998). Artificial neural networks for solving ordinary and partial differential equations. *IEEE Transactions on Neural Networks / a Publication of the IEEE Neural Networks Council*, *9*(5), 987–1000. https://doi.org/10.1109/72.712178

Lan, K. (2022). Dream fusion in octahedral spherical hohlraum. *Matter and Radiation at Extremes*, *7*(5). https://doi.org/10.1063/5.0103362

Mallat, S. G. (1989). A theory for multiresolution signal decomposition: the wavelet representation. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, *11*(7), 674–693. https://doi.org/10.1137/1.9781611970104

McClenny, L. D., & Braga-Neto, U. M. (2023). Self-adaptive physics-informed neural networks. *Journal of Computational Physics*, *474*, Paper No. 111722, 23. https://doi.org/10.1016/j.jcp.2022.111722

Moelans, N., Blanpain, B., & Wollants, P. (2008). An introduction to phase-field modeling of microstructure evolution. *CALPHAD ; Computer Coupling of Phase Diagrams and Thermochemistry*, *32*(2), 268–294. https://doi.org/10.1016/j.calphad.2007.11.003

Navaneeth, N., & Chakraborty, S. (2023). Stochastic projection based approach for gradient free physics informed learning. *Computer Methods in Applied Mechanics and Engineering*, *406*, Paper No. 115842, 21. https://doi.org/10.1016/j.cma.2022.115842

Rahaman, N., Baratin, A., Arpit, D., Draxler, F., Lin, M., Hamprecht, F., Bengio, Y., & Courville, A. (2019). On the spectral bias of neural networks. In *International conference on machine learning* (pp. 5301–5310). PMLR.

Raissi, M., Perdikaris, P., & Karniadakis, G. E. (2019). Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, *378*, 686–707. https://doi.org/10.1016/j.jcp.2018.10.045

Roos, H.-G., Stynes, M., & Tobiska, L. (2008). Robust numerical methods for singularly perturbed differential equations (vol. 24). Springer Series in Computational Mathematics. (2nd ed.). Springer-Verlag, Berlin. Convection-diffusion-reaction and flow problems. https://doi.org/10.1007/978-3-540-34467-4

Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, *323*(6088), 533–536. https://doi.org/10.1038/323533a0

Sharma, R., & Shankar, V. (2022). Accelerated training of physics-informed neural networks (pinns) using meshless discretizations. *NeurIPS*, *35*, 1034–1046. https://proceedings.neurips.cc/paper_files/paper/2022/file/0764db1151b936aca59249e2c1386101-Paper-Conference.pdf.

Sirignano, J., & Spiliopoulos, K. (2018). DGM: A deep learning algorithm for solving partial differential equations. *Journal of Computational Physics*, *375*, 1339–1364. https://doi.org/10.1016/j.jcp.2018.08.029

Su, J. (1993). Delayed oscillation phenomena in the fitzhugh nagumo equation. *Journal of Differential Equations*, *105*(1), 180–215. https://doi.org/10.1006/jdeq.1993.1087

Uddin, Z., Ganga, S., Asthana, R., & Ibrahim, W. (2023). Wavelets based physics informed neural networks to solve non-linear differential equations. *Scientific Reports*, *13*(1), 2882. https://doi.org/10.1038/s41598-023-29806-3

Wang, S., Teng, Y., & Perdikaris, P. (2021a). Understanding and mitigating gradient flow pathologies in physics-informed neural networks. *SIAM Journal of Scientific Computing*, *43*(5), A3055–A3081. https://doi.org/10.1137/20M1318043

Wang, S., Wang, H., & Perdikaris, P. (2021b). On the eigenvector bias of fourier feature networks: From regression to solving multi-scale PDEs with physics-informed neural networks. *Computer Methods in Applied Mechanics and Engineerin*, *384*, Paper No. 113938, 28. https://doi.org/10.1016/j.cma.2021.113938

Wang, S., Yu, X., & Perdikaris, P. (2022). When and why PINNs fail to train: A neural tangent kernel perspective. *Journal of Computational Physics*, *449*, Paper No. 110768, 28. https://doi.org/10.1016/j.jcp.2021.110768

Wang, Y., Yao, Y., Guo, J., & Gao, Z. (2024). A practical PINN framework for multi-scale problems with multi-magnitude loss terms. *Journal of Computational Physics*, *510*, Paper No. 113112, 19. https://doi.org/10.1016/j.jcp.2024.113112

Wight, C. L., & Zhao, J. (2021). Solving allen-cahn and cahn-hilliard equations using the adaptive physics informed neural networks. *Communications in Computational Physics*, *29*(3), 930–954. https://doi.org/10.4208/cicp.oa-2020-0086

Xiao, Y., Yang, L. M., Du, Y. J., Song, Y. X., & Shu, C. (2023). Radial basis function-differential quadrature-based physics-informed neural network for steady incompressible flows. *Physics of Fluids*, *35*(7). https://doi.org/10.1063/5.0159224

Yu, J., Lu, L., Meng, X., & Karniadakis, G. E. (2022). Gradient-enhanced physics-informed neural networks for forward and inverse PDE problems. *Computer Methods in Applied Mechanics and Engineerin*, *393*, Paper No. 114823, 22. https://doi.org/10.1016/j.cma.2022.114823